

Python: Recap IV

This notebook was generated in **solutions** mode.

Exercise 1: ASCII sum substring

Exercise Description

You are given a number N and a string S . In Python, a character can be converted to a numeric code (its ASCII representation) using the function `ord()`. For example, `ord('A')` is 65, `ord('B')` is 66, `ord('a')` is 97, `ord('1')` is 49, and so on.

Write a function `ascii_sum(N, S)` that returns `True` if S contains a substring whose **sum of ASCII codes** is exactly N , and `False` otherwise.

Example: if $N = 180$ and $S = \text{'CAB19'}$, the function must return `True` because there exists the substring `'AB1'` such that $\text{ord('A')} + \text{ord('B')} + \text{ord('1')} = 65 + 66 + 49 = 180$.

Your solution

```
def ascii_sum(N, S):
    for i in range(len(S)):
        for j in range(i + 1, len(S) + 1):
            this_substring = S[i:j]
            this_sum = sum(ord(letter) for letter in this_substring)
            if this_sum == N:
                return True
    return False
```

Test your solution

```
# Test case 1 (Example)
N = 180
S = 'CAB19'
assert ascii_sum(N, S) is True

# Test case 2
N = 180
S = 'CAB91'
assert ascii_sum(N, S) is False

# Test case 3
N = 200
S = 'Hello, World!'
assert ascii_sum(N, S) is False

# Test case 4
N = 65 # ASCII code for 'A'
S = 'ABCDEF'
assert ascii_sum(N, S) is True
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Flip if upper

Exercise Description

Write a function `my_function(s)` that receives a string `s` (a sentence where words are separated by whitespaces) and returns a new string built according to the following rule: if a word in `s` begins with an uppercase letter, then the word must be **reversed** in the new string; otherwise it must be left intact.

The order of the words between the input and the output string must not change. If the input string is empty, the function must return `None`.

Note: the examples include a trailing space at the end of the returned string.

Your solution

```

def my_function(s):
    # let's check if the input string is empty
    if s == '':
        return None

    # if not, then split the string into words
    lst = s.split()

    # initialize an empty string that we will fill with the (eventually reversed) words
    s_new = ''
    for word in lst:
        if word[0].isupper():
            s_new += word[::-1] + ' '
        else:
            s_new += word + ' '

    # job done
    return s_new

```

Test your solution

```

# Test case 1 (Example)
s = 'hello World'
t = my_function(s)
assert t == 'hello dlroW '

# Test case 2
s = ''
t = my_function(s)
assert t is None

```

```
# Test case 3
s = 'Python is AWESOME'
t = my_function(s)
assert t == 'nohtyP is EMOSEWA '

# Test case 4
s = 'viva david parenzo'
t = my_function(s)
assert t == 'viva david parenzo '
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Longest word in string

Exercise Description

Write a function `my_function(s)` that receives a string `s` (representing a sentence where words are separated by whitespaces) and returns the **longest word** in `s`.

If the input string is empty, the function must return `None`. If there are more than one longest words, the function must return the **first** one.

Your solution

```
def my_function(s):  
    # let's check if the input string is empty  
    if s == '':  
        return None  
  
    # if not, then split the string into words  
    lst = s.split()  
  
    # initialize variables to keep track of the longest word  
    max_length = 0  
  
    # iterate over the words  
    for word in lst:  
        # do we have a new longest word?  
        if len(word) > max_length:  
            max_length = len(word)  
            longest_word = word  
  
    return longest_word
```

Test your solution

```
# Test case 1 (Example)  
s = 'This is an example sentence with some long words.'  
assert my_function(s) == 'sentence'
```

```
# Test case 2
s = 'The quick brown fox jumps over the lazy dog.'
assert my_function(s) == 'quick'

# Test case 3
s = ''
assert my_function(s) is None

# Test case 4
s = 'Ragazzi ma se poi senti Parento su lasette che fa il grande storico'
assert my_function(s) == 'Ragazzi'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Periodic String Group

Exercise Description

Write a function `my_function(X, Y, S)` that, given two integer values `X` and `Y` and a string `S` of length 1 (a character), returns a string with `Y` **groups** of characters. Each group consists of `X` copies of `S`. Groups are separated by `' - '`.

You can assume that `X` and `Y` are greater than or equal to 0. If `X == 0`, groups are empty and no `' - '` separators should be used (the function should return the empty string).

Hint: you can use the `*` operator to obtain sequences of equal characters, e.g., `'x' * 7 == 'xxxxxxx'`.

Your solution

```
def my_function(X, Y, S):
    new_s = ''
    if X == 0:
        return new_s
    for i in range(Y - 1):
        new_s = new_s + (X * S) + ' - '
    new_s = new_s + (X * S)
    return new_s
```

Test your solution

```
# Test case 1 (Example)
s1 = my_function(4, 2, 'A')
assert s1 == 'AAAA-AAAA'

# Test case 2
s1 = my_function(1, 10, 'A')
```



```
assert s1 == 'A-A-A-A-A-A-A-A-A-A'

# Test case 3
s1 = my_function(0, 5, 'A')
assert s1 == ''

# Test case 4
s1 = my_function(3, 5, 'A')
assert s1 == 'AAA-AAA-AAA-AAA-AAA'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Periodic String

Exercise Description

Write a function `my_function(X, Y, S)` that, given two integer values `X` and `Y` and a string `S` of length 1 (a character), returns a string with `X` characters `S` separated by `Y` characters `' - '`. You can assume that `X` and `Y` are greater than or equal to 0.

If $X \neq 1$ no '-' characters should be added, independently of the value of Y .

Hint: you can use the $*$ operator to obtain sequences of equal characters (e.g., $'x' * 7 == 'xxxxxxx'$).

Your solution

```
def my_function(X, Y, S):
    new_s = ''
    if X != 0:
        for i in range(X - 1):
            new_s = new_s + S + (Y * '-')
        new_s = new_s + S
    return new_s
```

Test your solution

```
# Test case 1 (Example)
s1 = my_function(3, 2, 'A')
assert s1 == 'A--A--A'

# Test case 2
s1 = my_function(1, 10, 'A')
assert s1 == 'A'

# Test case 3
s1 = my_function(0, 5, 'A')
assert s1 == ''

# Test case 4
```

```
s1 = my_function(4, 5, 'A')  
assert s1 == 'A-----A-----A-----A'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Remove Char

Exercise Description

Write a function `my_function(S, C)` that, given as parameters a string `S` and another string `C` of length 1 (a character), returns a string `T` that is equal to `S` except that it contains **no occurrence** of character `C`.

Your solution

```
def my_function(S, C):  
    new_s = ''  
    for letter in S:  
        if letter != C:
```

```
        new_s = new_s + letter
    return new_s
```

Test your solution

```
# Test case 1 (Example)
s = 'I study at Luiss'
c = 's'
t = my_function(s, c)
assert t == 'I tudy at Lui'

# Test case 2
s = 'xxxxxxo'
c = 'x'
t = my_function(s, c)
assert t == 'o'

# Test case 3
s = 'xxxxxxxx 000'
c = 'x'
t = my_function(s, c)
assert t == 'o 000'

# Test case 4
s = 'xxxxxxxxo'
c = 's'
t = my_function(s, c)
assert t == 'xxxxxxxxo'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Replace Substring

Exercise Description

Write a function `my_function(S, R, C)` that, given as parameters a string `S`, a string `R` and another string `C` of length 1 (a character), returns a string `T` that is equal to `S` except that each occurrence of character `C` is **replaced** by string `R`.

Your solution

```
def my_function(S, R, C):
    new_s = ''
    for letter in S:
        if letter == C:
            new_s = new_s + R
        else:
            new_s = new_s + letter
    return new_s
```

Test your solution

```
# Test case 1 (Example)
s = 'I study at Luiss'
r = 'REPLACE'
c = 's'
t = my_function(s, r, c)
assert t == 'I REPLACEstudy at LuiREPLACEREPLACE'

# Test case 2
s = 'I REPLACEddd'
r = 'REPLACE'
c = 'D'
t = my_function(s, r, c)
assert t == 'I REPLACEddd'

# Test case 3
s = 'D'
r = 'REPLACE'
c = 'D'
t = my_function(s, r, c)
assert t == 'REPLACE'

# Test case 4
s = ''
r = 'REPLACE'
c = 'D'
t = my_function(s, r, c)
assert t == ''
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Driving License

Exercise Description

Write a function `my_function(license_plate)` that validates a driving license plate format.

The function should return `True` if the license plate follows the correct format, and `False` otherwise.

A valid license plate must:

- Be exactly 7 characters long
- Have exactly 4 letters followed by exactly 3 digits
- All letters must be uppercase

For example: "ABCD123" is valid, but "ABC123", "ABCDE123", or "ABCD12" are not valid.

Your solution

```
def my_function(license_plate):  
    if len(license_plate) != 7:  
        return False  
    if not license_plate[:4].isalpha() or not license_plate[:4].isupper():  
        return False  
    if not license_plate[4:].isdigit():  
        return False  
    return True
```

Test your solution

```
# Test case 1 (Example)  
result = my_function("ABCD123")  
assert result == True  
  
# Test case 2  
result = my_function("ABC123")  
assert result == False  
  
# Test case 3  
result = my_function("ABCDE123")  
assert result == False  
  
# Test case 4  
result = my_function("ABCD12")  
assert result == False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Reverse Order

Exercise Description

Write a Python function `my_function(s)` that takes a string `s` as an argument and returns a new **list of words** that contains the words in `s` in reverse order.

For example, if `s = 'I love you'`, then `my_function(s)` should return `['you', 'love', 'I']`.

Hint: you can use the `split()` method to split the string into a list of words.

Your solution

```
def my_function(s):  
    s = s.split()  
    return s[::-1]
```

Test your solution

```
# Test case 1 (Example)
s1 = 'I love you'
assert my_function(s1) == ['you', 'love', 'I']

# Test case 2
s2 = 'The quick brown fox'
assert my_function(s2) == ['fox', 'brown', 'quick', 'The']

# Test case 3
s3 = 'Hello World'
assert my_function(s3) == ['World', 'Hello']

# Test case 4
s4 = 'Python is fun'
assert my_function(s4) == ['fun', 'is', 'Python']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Alien race

Exercise Description

It's 2027. Earth has been invaded by three alien races: the Greys, the Reptilians, and the Annunaki. A running race is organized to decide which race will dominate. At the end of the race, a list is provided with the order in which the participants finished. Each element of the list is in the format participant - race.

The score of a race is determined by **adding the positions** of all its participants, and the race with the **lowest score** wins.

Write a function `my_function(order_of_arrival, race)` that calculates the score obtained by a specific race. Positions are 1-based (the first element has position 1). If the list does not include that race among the participants, the result should be 0.

Your solution

```
def my_function(order_of_arrival, race):
    score = 0
    i = 0
    for el in order_of_arrival:
        i += 1
        r = el.split('-')[1]
        if r == race:
            score += i
    return score
```

Test your solution

```
# Test case 1
order_of_arrival = ['Gix-Annunaki', 'Tom-Grey', 'XYZ-Reptilian', 'Alex-Annunaki', 'Ar
race = 'Annunaki'
assert my_function(order_of_arrival, race) == 22

# Test case 2
race = 'Grey'
assert my_function(order_of_arrival, race) == 26

# Test case 3
order_of_arrival = ['Gix-Annunaki', 'Tom-Grey', 'Alex-Annunaki', 'AngelFab-Grey', 'Mi
race = 'Reptilian'
assert my_function(order_of_arrival, race) == 0

# Test case 4
order_of_arrival = ['Gix-Annunaki', 'Tom-Grey', 'XYZ-Reptilian', 'Alex-Annunaki', 'Ar
race = 'Humans'
assert my_function(order_of_arrival, race) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 11: Remove numeric duplicates and sort

Exercise Description

Write a function `my_function(L)` that, given a list of numbers `L`, returns a new list containing the same numbers **without duplicates**.

The numbers in the output list must be sorted from the smallest to the largest. If the input list is empty, the function should return an empty list.

Your solution

```
def my_function(L):  
    L_new = []  
    for item in L:  
        if item not in L_new:  
            L_new.append(item)  
    L_new = sorted(L_new)  
    return L_new
```

Test your solution

```
# Test case 1 (Example)  
assert my_function([10, 2, 3, 5, 2, 10, 3]) == [2, 3, 5, 10]  
  
# Test case 2  
assert my_function([]) == []
```

```
# Test case 3
assert my_function([1, 1, 1, 1, 1, 1]) == [1]

# Test case 4
assert my_function([1, -2, 3, -4, 5, -6, 7, -8, 9, -10] * 1000) == [-10, -8, -6, -4,
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: Euro To Dollars

Exercise Description

Write a function `my_function(L)` that is given a list `L` of numbers representing coins in euro. The function should return the **sum of the coins converted to dollars**, where 1 euro corresponds to 1.22 dollars.

If there are no items in the list, the function should return `0.0` (a float).

Your solution

```
def my_function(L):  
    s = 0  
    for n in L:  
        s = s + n * 1.22  
    return s  
    # return sum([item * 1.22 for item in L])
```

Test your solution

```
# Test case 1 (Example)  
assert my_function([3, 2, 4]) == 10.98  
  
# Test case 2  
assert my_function([0, 0, 1]) == 1.22  
  
# Test case 3  
assert my_function([2, 5, 12]) == 23.18  
  
# Test case 4  
assert float(my_function([])) == 0.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: H-index

Exercise Description

The h -index is a common index to quantify the scientific impact of a researcher. For a given researcher, you are given a list papers of n integer numbers, where n is the number of papers. In position i you have the number of citations of the i -th paper.

The h -index is defined as the largest number h such that there are **at least h papers with at least h citations**.

Write a function `my_function(papers)` that returns the h -index of the researcher. You can assume the list contains only integers. If the input list is empty, the function must return `None`.

Your solution

```
def my_function(citations):  
    n = len(citations)  
    if n == 0:  
        return None  
    citations_srt = sorted(citations, reverse=True)  
    h = 0  
    for i in range(n):  
        if citations_srt[i] >= i + 1:
```



```
        h += 1
    return h
```

Test your solution

```
# Test case 1 (Example)
assert my_function([24, 2, 36, 2, 3]) == 3

# Test case 2
assert my_function([3, 0, 6, 1, 5]) == 3

# Test case 3
assert my_function([]) is None

# Test case 4
assert my_function([24, 6, 12, 36, 2, 2]) == 4
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 14: Km To Miles

Exercise Description

Write a function `my_function(L)` that is given a list `L` of numbers representing distances in kilometers. The function should return the **sum of the distances converted to miles**, where 1 kilometer corresponds to 0.62 miles.

If there are no items in the list, the function should return 0.

Your solution

```
def my_function(L):  
    return sum([item * 0.62 for item in L])
```

Test your solution

```
# Test case 1 (Example)  
assert my_function([3, 2, 4]) == 5.58  
  
# Test case 2  
assert my_function([3]) == 1.8599999999999999  
  
# Test case 3  
assert float(my_function([0])) == 0.0  
  
# Test case 4  
assert float(my_function([])) == 0.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 15: Name Ends With

Exercise Description

Write a function `my_function(L1, L2)` that takes two lists as parameters:

- each element in `L1` is the name of a person,
- each element in `L2` is a character (a string of length 1).

The function shall return a list `L3` containing the persons whose name **ends with** a letter in `L2`. Elements in `L3` should appear in the same order in which they appear in `L1`.

If `L1` is empty or does not contain any person whose name ends with a letter in `L2`, then `L3` must be an empty list.

Your solution

```
def my_function(L1, L2):  
    L3 = []
```

```
for name in L1:
    for letter in L2:
        if name.endswith(letter):
            L3.append(name)
            break
return L3
```

Test your solution

```
# Test case 1 (Example)
assert my_function(["Giorgio", "Irene", "Antonio", "Carla"], ["o", "e"]) == ["Giorgio"]

# Test case 2
assert my_function(["Giorgio", "Irene", "Antonio", "Carla"], ["e", "x"]) == ["Irene"]

# Test case 3
assert my_function(["Giorgio", "Irene", "Antonio", "Carlx"], ["e", "x"]) == ["Irene", "Carlx"]

# Test case 4
assert my_function(["Giorgio", "Irene", "Antonio", "Carlx"], ["v", "z"]) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 16: Nevio the Ironed

Exercise Description

"Nevio the Ironed" is a fervent gambler who has accumulated a series of debts with banks. He is ordered to repay the amounts owed. Nevio currently has in his pocket a `winning_sum` of money he just won by betting on a horse.

You are given a list `debts` where each element has the format `'amount - repaid'`:

- `amount` is an integer amount of money,
- `repaid` is 0 if the amount has **not** yet been settled and 1 if it has already been repaid.

Write a function `my_function(debts, winning_sum)` that returns `True` if Nevio can settle **all unpaid debts** using `winning_sum`, and `False` otherwise.

Your solution

```
def my_function(debts, winning_sum):  
    for debt in debts:  
        if debt[-1] == '0':  
            winning_sum -= int(debt[:-2])  
    return winning_sum >= 0
```

Test your solution

```
# Test case 1
debts = ['300-1', '200-0', '100-0']
winning_sum = 500
assert my_function(debts, winning_sum) is True

# Test case 2
debts = ['500-0', '200-0', '100-1']
winning_sum = 500
assert my_function(debts, winning_sum) is False

# Test case 3
debts = ['500-0', '200-0', '100-1']
winning_sum = 400
assert my_function(debts, winning_sum) is False

# Test case 4
debts = ['500-0', '200-0', '100-1']
winning_sum = 300
assert my_function(debts, winning_sum) is False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 17: No escape for Earth

Exercise Description

Some theories suggest the existence of a planet called Nibiru or Planet X. Scientists have identified a series of meteors approaching Earth. For each meteor, they recorded its radius and whether its trajectory is directed towards Earth.

Data are given in a list `data`, where each element has the format 'radius-trajectory':

- `radius` is a number,
- `trajectory` is 0 if it is **not** directed towards Earth, and 1 otherwise.

You are also given:

- `circumference`: the **maximum acceptable circumference** for a meteor,
- `limit`: the **maximum number of meteors** directed towards Earth that we can withstand.

Write a function `my_function(data, circumference, limit)` that returns:

- `False` if **any** meteor has circumference greater than `circumference`, or if the number of meteors with trajectory 1 is greater than `limit`,
- `True` otherwise.

You can use `math.pi` and the formula `circ = radius * radius * pi`.

Your solution

```
import math

def my_function(data, circumference, limit):
    count = 0
    for i in data:
        i = i.split('-')
        if float(i[0]) * float(i[0]) * math.pi > circumference:
            return False
        if i[1] == '1':
            count += 1
    if count > limit:
        return False
    else:
        return True
```

Test your solution

```
# Test case 1
data = ['100-1', '200-1', '300-0', '400-1', '500-0']
circumference = 400
limit = 2
assert my_function(data, circumference, limit) is False

# Test case 2
limit = 1
assert my_function(data, circumference, limit) is False

# Test case 3
```



```
limit = 3
assert my_function(data, circumference, limit) is False

# Test case 4
circumference = 3000000
limit = 4
assert my_function(data, circumference, limit) is True
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 18: Person and first letter

Exercise Description

Write a function `my_function(L1, L2)` that takes two lists as parameters:

- each element in list `L1` is the name of a person,
- each element in list `L2` is a character (a string of length 1).

The function shall return a list L3 containing the persons whose name **begins with** a letter in L2. Elements in L3 should appear in the same order in which they appear in L1.

If L1 is empty or does not contain persons whose name begins with a letter in L2, then L3 must be an empty list.

Your solution

```
def my_function(L1, L2):  
    L3 = []  
    for name in L1:  
        initial = name[0]  
        if initial in L2:  
            L3.append(name)  
    return L3
```

Test your solution

```
# Test case 1 (Example)  
assert my_function(['Giorgio', 'Irene', 'Antonio', 'Giuseppe'], ['I', 'G']) == ['Giorgio', 'Giuseppe']  
  
# Test case 2  
assert my_function(['Giorgio', 'Mino', 'Antonio', 'Marco'], ['I', 'M']) == ['Mino', 'Marco']  
  
# Test case 3  
assert my_function(['Giorgio', 'Mino', 'Antonio', 'Marco'], ['I', 'S']) == []  
  
# Test case 4  
assert my_function([], ['I', 'M']) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 19: Products of Elements

Exercise Description

Write a function `my_function(L1, L2)` that takes as parameters two lists of integer numbers.

The function shall return an empty list if `L1` and `L2` are empty **or** do not have the same length. Otherwise, it shall return a list `L3` containing the products of elements in the same position in `L1` and `L2`.

Your solution

```
def my_function(L1, L2):  
    L3 = []  
    # do not iterate if lists are empty  
    if len(L1) == 0 and len(L2) == 0:
```

```
        return L3
    # do not iterate if they have different lengths
    if len(L1) != len(L2):
        return L3
    # otherwise, do
    for i in range(len(L2)):
        L3.append(L2[i] * L1[i])
    return L3
```

Test your solution

```
# Test case 1 (Example)
assert my_function([10, 20, 30], [100, 200, 300]) == [1000, 4000, 9000]

# Test case 2
assert my_function([10, 30], [100, 300]) == [1000, 9000]

# Test case 3
assert my_function([10, 30, 15], [100, 300]) == []

# Test case 4
assert my_function([], []) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 20: Remove duplicates (strings)

Exercise Description

Define a function `my_function(lst)` that takes a list of strings `lst` as input.

The function shall return a list containing the same elements of `lst`, but **without duplicates**: if a string `s` appears more than once in `lst`, only the **first** occurrence of `s` should be copied to the output list.

If the input list `lst` is empty, the function should return an empty list.

Your solution

```
def my_function(lst):  
    output = []  
    for item in lst:  
        if item not in output:  
            output.append(item)  
    return output
```

Test your solution

```

# Test case 1 (Example)
a = ['a', 'good', 'day', 'is', 'good', 'a', 'bad', 'day', 'is', 'not', 'good']
assert my_function(a) == ['a', 'good', 'day', 'is', 'bad', 'not']

# Test case 2
a = ['test', 'test', 'test', 'test', 'test', 'test', 'test', 'test']
assert my_function(a) == ['test']

# Test case 3
a = ['test', 'test', 'hello', 'test', 'hello', 'test', 'hello', 'test']
assert my_function(a) == ['test', 'hello']

# Test case 4
a = []
assert my_function(a) == []

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 21: Reptilians

Exercise Description

The Reptilians are among us! An intelligence agency has revealed that many presidents of the world's nations are actually clones controlled by the Reptilians. The clones have specific characteristics:

- They have odd-length names.
- They have light-colored eyes.
- They are over 40 years old.

Each president is represented as a string in the format 'name-eyes-age', where eyes can be either 'light' or 'dark'.

Write a function `my_function(presidents)` that returns the number of presidents that are clones according to the rules above.

Your solution

```
def my_function(presidents):  
    number = 0  
    for president in presidents:  
        p_split = president.split('-')  
        name = p_split[0]  
        eyes = p_split[1]  
        age = p_split[2]  
        if len(name) % 2 == 1 and eyes == 'light' and int(age) > 40:  
            number += 1  
    return number
```

Test your solution

```
# Test cases 1  
presidents = ['John-light-45', 'George-dark-54', 'Barack-dark-55', 'Bill-light-68', 'F  
assert my_function(presidents) == 1  
  
# Test cases 2  
presidents = ['John-light-45', 'George-dark-54', 'Barack-dark-55', 'Bill-light-68', 'F  
assert my_function(presidents) == 2  
  
# Test cases 3  
presidents = ['John-light-40', 'George-dark-54', 'Barack-dark-55', 'Bill-dark-68', 'F  
assert my_function(presidents) == 0  
  
# Test cases 4  
presidents = []  
assert my_function(presidents) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 22: Rugby betting scandal

Exercise Description

In New Zealand, a rugby betting scandal has erupted. Some players have placed bets on certain matches despite the rules prohibiting it. The rules forbid betting from any **illegal** platform, and there is also a maximum amount of money that can be spent on legal platforms.

The federation has gathered a list `bets` of bets placed by players, where each element has the format `'name-bet-illegal'`:

- `name` is the name of the rugby player,
- `bet` is the amount wagered (a float),
- `illegal` is 0 if the site was legal, and 1 otherwise.

Write a function `my_function(bets, player)` that returns `True` if the player should be **disqualified**, and `False` otherwise.

A player is disqualified if:

- they placed **any** bet on an illegal platform, or
- the **total** amount they bet on legal platforms exceeds 500.

Your solution

```
def my_function(bets, player):  
    total = 0  
    for bet in bets:  
        bet = bet.split('-')  
        if bet[0] == player:  
            if bet[2] == '1':  
                return True  
            else:  
                total += float(bet[1])  
    if total > 500:  
        return True  
    return False
```

Test your solution

```
# Test case 1  
bets = ['Beans-30.5-0', 'Beans-20.4-1', 'Tonal-50.5-1', 'Tonal-100.5-0', 'TheShara-21  
player = 'Beans'  
assert my_function(bets, player) is True  
  
# Test case 2  
player = 'Tonal'  
assert my_function(bets, player) is True
```

```
# Test case 3
player = 'TheShara'
assert my_function(bets, player) is False

# Test case 4
player = 'Mikasa'
assert my_function(bets, player) is False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 23: Subtract digit sum

Exercise Description

Write a function `subtract_digits(n)` that, given an integer number `n`, returns the number `d` obtained by **subtracting from `n` the sum of its digits**.

Hint: convert `n` into a string and work on each character, which corresponds to a digit.

Example: if $n = 12$, the function should return 9 because $12 - 1 - 2 = 9$.

Your solution

```
def subtract_digits(n):  
    n_s = str(n)  
    for digit in n_s:  
        n = n - int(digit)  
    return n
```

Test your solution

```
# Test case 1 (Example)  
assert subtract_digits(12) == 9  
  
# Test case 2  
assert subtract_digits(1038) == 1026  
  
# Test case 3  
assert subtract_digits(0) == 0  
  
# Test case 4  
assert subtract_digits(1199) == 1179
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 24: Sum larger or equal 3

Exercise Description

Write a function `my_function(L)` that, given a list `L` of strings, returns the **sum of the lengths** of the strings whose length is **greater than or equal to 3**.

If there are no items in the list or no strings of length ≥ 3 , the function should return 0.

Your solution

```
def my_function(L):  
    s = 0  
    for string in L:  
        if len(string) >= 3:  
            s = s + len(string)  
    return s
```

Test your solution

```
# Test case 1 (Example)  
assert my_function(['luiss', 'row', 'a', 'bx']) == 8  
  
# Test case 2
```

```
assert my_function(['a', 'a', 'a', 'a']) == 0

# Test case 3
assert my_function(['aa', 'aaaa', 'aaa', 'aaa']) == 10

# Test case 4
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 25: Sum of Even Squares

Exercise Description

Write a function `my_function(L)` that, given a list `L` of numbers, returns the **sum of the squares of the even numbers** contained in `L`.

If there are no items or no even numbers in the list, the function should return `0`.

Your solution

```
def my_function(L):  
    # check if the list is empty  
    if L == []:  
        return 0  
    # even numbers  
    even_numbers = [x for x in L if x % 2 == 0]  
    if even_numbers == []:  
        return 0  
    # square of even numbers  
    return sum([x ** 2 for x in even_numbers])
```

Test your solution

```
# Test case 1 (Example)  
assert my_function([2, 13, 8, 5]) == 68  
  
# Test case 2  
assert my_function([1, 3, 5, 7]) == 0  
  
# Test case 3  
assert my_function([3, 101, 9, 7, 4]) == 16  
  
# Test case 4  
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 26: Sum of all the odd

Exercise Description

You have an integer number n as input. Define a function `all_odd(n)` that returns the **sum of all odd integer numbers between 1 and n** , n included.

Your solution

```
def all_odd(n):  
    c = 0  
    for i in range(1, n + 1):  
        if i % 2 == 0:  
            pass  
        else:  
            c = c + i  
    return c
```

Test your solution

```
# Test case 1 (Example)  
assert all_odd(7) == 16
```



```
# Test case 2
assert all_odd(6) == 9

# Test case 3
assert all_odd(100) == 2500

# Test case 4
assert all_odd(1000 * 1000) == 2500000000000
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 27: Valid votes

Exercise Description

You are given a list of votes `votes` and a candidate name `candidate`. Each vote is represented as a string in the format `'candidate-valid_vote'`, where `valid_vote` is either 0 or 1 (1 means the vote is valid).

Write a function `my_function(votes, candidate)` that **counts how many valid votes** (with `valid_vote == '1'`) the given candidate received and returns that count. The candidate may have received 0 votes.

Your solution

```
def my_function(votes, candidate):  
    valid_votes = 0  
    for vote in votes:  
        if vote.split('-')[0] == candidate and vote.split('-')[1] == '1':  
            valid_votes += 1  
    return valid_votes
```

Test your solution

```

# Test case 1
votes = ['A-1', 'A-0', 'B-1', 'B-1', 'C-0', 'A-1', 'C-1', 'D-0']
assert my_function(votes, 'A') == 2

# Test case 2
votes = ['A-0', 'B-1', 'C-1', 'D-1', 'E-0', 'F-1', 'G-1', 'H-0']
assert my_function(votes, 'H') == 0

# Test case 3
votes = ['A-1', 'B-1', 'C-1', 'D-1', 'E-1', 'F-1', 'G-1', 'H-1']
assert my_function(votes, 'A') == 1

# Test case 4
votes = ['A-0', 'B-0', 'C-0', 'D-0', 'E-0', 'F-0', 'G-0', 'H-0']
assert my_function(votes, 'A') == 0

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 28: Vowels

Exercise Description

You have a string `a` as input, along with two integer numbers `s` and `e`. Define a function `my_function(a, s, e)` that returns **how many vowels** are in the substring starting at position `s` and ending at position `e` (both included).

Consider the vowels `a, e, i, o, u` in both lowercase and uppercase.

Your solution

```
def my_function(a, s, e):  
    a = a[s: e + 1]  
    a = a.lower()  
    c = 0  
    for letter in a:  
        if letter == 'a' or letter == 'e' or letter == 'i' or letter == 'o' or letter == 'u':  
            c = c + 1  
    return c
```

Test your solution

```
# Test case 1 (Example)  
a = 'I study at Luiss'  
assert my_function(a, 0, 15) == 5  
  
# Test case 2  
a = 'aaccocoa'  
assert my_function(a, 0, 7) == 4
```

```
# Test case 3
a = 'AAAAmmm'
assert my_function(a, 4, 6) == 0

# Test case 4
a = 'xyz rtw lpq'
assert my_function(a, 1, 7) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 29: Aggregate Address

Exercise Description

Write a function `my_function(addresses)` that aggregates addresses by town.

`addresses` is a list of tuples, where each tuple has two string elements: `(town, street_name)`. For example, it could be `addresses = [('Roma', 'Via Panama'), ('Roma', 'Viale Romania'), ('Milano', 'Corso Sempione')]`.

The function shall return a dictionary that maps each town to the number of addresses in that town from the input list. If the input list is empty, the function should return an empty dictionary.

Your solution

```
def my_function(addresses):
    aggregator = {}
    # do not iterate if list is empty
    if len(addresses) == 0:
        return aggregator
    # otherwise, do
    for item in addresses:
        city = item[0]
        # address = item[1]
        if city in aggregator:
            aggregator[city] += 1
        else:
            aggregator[city] = 1
    return aggregator
```

Test your solution

```
# Test case 1 (Example)
a = my_function([('Roma', 'Via Panama'), ('Roma', 'Viale Romania'), ('Milano', 'Corso')])
assert a == {'Roma': 2, 'Milano': 1}

# Test case 2
a = my_function([('Londra', 'Via Panama'), ('Milano', 'Viale Romania'), ('Milano', 'Corso')])
assert a == {'Londra': 1, 'Milano': 2}
```

```
# Test case 3
a = my_function([])
assert a == {}

# Test case 4
a = my_function([('Firenze', 'Via Panama'), ('Como', 'Viale Romania'), ('Napoli', 'Co
assert a == {'Firenze': 1, 'Como': 1, 'Napoli': 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 30: Deck

Exercise Description

Write a function `my_function(deck)` that processes a deck of cards represented as a dictionary.

The `deck` parameter is a dictionary where keys are card names (strings) and values are the number of cards of that type. The function should return a new dictionary containing only the

cards that appear more than once in the deck.

If no cards appear more than once, return an empty dictionary.

Your solution

```
def my_function(deck, points):  
    # do not iterate if deck is empty  
    if len(deck) == 0:  
        return 0  
    # otherwise, do  
    s = 0  
    for card in deck:  
        if card in points:  
            s = s + points[card]  
    return s
```

Test your solution

```
# Test case 1 (Example)  
a = my_function(['Q', '6', 'A', '8'], {'A': 1, '8': 8, 'J': 11, 'Q': 12, 'K': 13})  
# assert a == expected_result # TODO: Add expected result  
  
# Test case 2  
a = my_function(['Q', '6', 'A'], {'8': 8, 'J': 11, 'Q': 12, 'K': 13})  
# assert a == expected_result # TODO: Add expected result  
  
# Test case 3  
a = my_function([], {'A': 1, '8': 8, 'J': 11, 'Q': 12, 'K': 13})  
# assert a == expected_result # TODO: Add expected result
```



```
# Test case 4
a = my_function(['Q', '6', 'A', '8'], {'K': 1, 'J': 11, '3': 3, '2': 13})
# assert a == expected_result # TODO: Add expected result
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 31: Dict Game 1

Exercise Description

Let D be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let L be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary D and a list L , returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```
def checkUppercase(word):  
    uppercase = "ABCDEFGHILMNOPQRSTUVWXYZ"  
    for letter in uppercase:  
        if letter in word:  
            return True  
    return False  
  
def my_function(D,L):  
    s = 0  
    for el in L:  
        if el in D:  
            if checkUppercase(el) and D[el] % 2 ==0:  
                s += len(el)  
    return s
```

Test your solution

```
# Test case 1 (Example)  
D = {"Sara" : 8, "Mara" : 3}  
L = ["Alessio", "Sara", "Giorgio", "Mara"]  
result = my_function(D, L)
```

```

assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Iaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Iaria"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Iaria" : 7}
L = []
result = my_function(D, L)
assert result == 0

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 32: Dict Game 2

Exercise Description

Let D be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let L be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary D and a list L , returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```
def my_function(D,L):  
    count = 0  
    for el in L:  
        if el in D:  
            if len(el) > 5 and D[el] % 2 == 0:  
                count += 1  
    return count
```

Test your solution

```
# Test case 1 (Example)
D = {"Sara" : 8, "Mara" : 3}
L = ["Alessio", "Sara", "Giorgio", "Mara"]
result = my_function(D, L)
assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilenia" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilenia"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Ilenia" : 7}
L = []
result = my_function(D, L)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 33: Dict Game 3

Exercise Description

Let D be a dictionary whose keys are words (strings) and whose corresponding values are dates. Further, let L be a list of words and let A and B be two `datetime.date` objects where $A < B$ (or A comes before B).

Write a function `my_function(D, L, A, B)` that, given as input a dictionary D , a list L and the two date objects A and B , returns the number of words (keys) in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least one uppercase letter
- The date associated with the word must be between A and B

If the input list is empty, the function must return 0.

Hint: Remember that it is possible to use the common operators for comparison (e.g., `<`, `>`, `==`) on date objects.

Recall: To use the `datetime` module in your code, please make sure to import it before the `def` of your function.

Your solution

```
import datetime

def checkUppercase(word):
    uppercase = "ABCDEFGHILMNOPQRSTUVWXYZ"
    for letter in uppercase:
        if letter in word:
```

```

        return True
    return False

def my_function(D, L, A, B):
    count = 0

    if L == []:      # check if the input list is empty: if so, return 0
        return 0

    for key in D.keys():
        if key in L:      # if the word is in the list
            if checkUppercase(key):      # if the word has at least one uppercase letter
                if A <= D[key] <= B:      # if the date is between A and B
                    count += 1

    return count

```

Test your solution

```

# Test case 1 (Example)
import datetime
D = {
    "apple": datetime.date(2023, 3, 15),
    "Banana": datetime.date(2023, 4, 10),
    "Orange": datetime.date(2022, 12, 5),
    "Grapes": datetime.date(2023, 5, 20),
}
L = ["apple", "Banana", "Cherry", "Grapes"]
A = datetime.date(2023, 1, 1)
B = datetime.date(2023, 6, 30)
result = my_function(D, L, A, B)
assert result == 2

```


Test case 2

```
D = {
    "Hello": datetime.date(2023, 3, 15),
    "World": datetime.date(2023, 4, 10),
    "Python": datetime.date(2022, 12, 5),
    "Programming": datetime.date(2023, 5, 20),
}
L = ["Hello", "Programming", "AI", "MachineLearning"]
A = datetime.date(2023, 1, 1)
B = datetime.date(2023, 6, 30)
result = my_function(D, L, A, B)
assert result == 2
```

Test case 3

```
D = {
    "Car": datetime.date(2023, 3, 15),
    "Bus": datetime.date(2023, 4, 10),
    "Bicycle": datetime.date(2022, 12, 5),
    "Train": datetime.date(2023, 5, 20),
}
L = ["Car", "Bus", "Bicycle", "Train"]
A = datetime.date(2023, 3, 1)
B = datetime.date(2023, 4, 30)
result = my_function(D, L, A, B)
assert result == 2
```

Test case 4

```
D = {
    "Sun": datetime.date(2023, 3, 15),
    "Moon": datetime.date(2023, 4, 10),
```

```
"Stars": datetime.date(2022, 12, 5),
"Planets": datetime.date(2023, 5, 20),
}
L = ["Sun", "Moon", "Stars", "Planets"]
A = datetime.date(2024, 1, 1)
B = datetime.date(2024, 6, 30)
result = my_function(D, L, A, B)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 34: Dict Game 4

Exercise Description

Let `D` be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let `L` be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary `D` and a list `L`, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```
def my_function(D, L):  
    count = 0  
    for el in L:  
        k, v = el  
        if k in D and (D[k] + v) < 100:  
            count += 1  
    return count
```

Test your solution

```
# Test case 1 (Example)  
D = {"Sara" : 8, "Mara" : 3}  
L = ["Alessio", "Sara", "Giorgio", "Mara"]  
result = my_function(D, L)  
assert result == 4  
  
# Test case 2  
D = {"Sara" : 8, "ilaria" : 4}  
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]  
result = my_function(D, L)  
assert result == 4
```

```
# Test case 3
D = {"Sara" : 8, "Ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilaria"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Ilaria" : 7}
L = []
result = my_function(D, L)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 35: Dict Game 5

Exercise Description

Let D be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let L be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary D and a list L, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```
def my_function(D, L):  
    s = 0  
    for el in L:  
        k, v = el  
        if k in D and (D[k] + v) % 2 == 1:  
            s += 1  
    return s
```

Test your solution

```
# Test case 1 (Example)  
D = {"Sara" : 8, "Mara" : 3}  
L = ["Alessio", "Sara", "Giorgio", "Mara"]  
result = my_function(D, L)
```

```

assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilarla" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilarla"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Ilarla" : 7}
L = []
result = my_function(D, L)
assert result == 0

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 36: Encrypt

Exercise Description

Write a function `my_function(text, key)` that encrypts `text` using a simple substitution cipher.

The `key` parameter is a dictionary that maps each character to its encrypted replacement. The function should return the encrypted version of the input text.

If a character in the text is not found in the `key` dictionary, it should remain unchanged in the output.

Your solution

```
def my_function(S, D):  
    result = ""  
    for el in S:  
        if el in D:  
            result += D[el]  
        else:  
            result += el  
    return result
```

Test your solution

```
# Test case 1 (Example)  
a = my_function("hvssm", {"v": "e", "t": "w", "s": "l", "m": "o"})
```

```

assert a == "hello"

# Test case 2
a = my_function("vtmx", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == "ewox"

# Test case 3
a = my_function("xxxxqqqq", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == "xxxxqqqq"

# Test case 4
a = my_function("", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == ""

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 37: Football League

Exercise Description

Write a function `my_function(teams, results)` that calculates the points for each team in a football league.

Parameters:

- `teams`: A list of team names (strings)
- `results`: A list of tuples, where each tuple represents a match result: (`team1`, `team2`, `goals1`, `goals2`)

Scoring rules:

- Win: 3 points
- Draw: 1 point
- Loss: 0 points

The function should return a dictionary mapping each team name to their total points.

Your solution

```
def my_function(matches):  
    teams = []  
    for match in matches:  
        teams.append(match[0])  
        teams.append(match[1])  
  
    D = {team: 0 for team in teams}
```

```

for match in matches:
    team1 = match[0]
    team2 = match[1]
    score1 = match[2]
    score2 = match[3]
    if score1==score2:
        D[team1] += 1
        D[team2] += 1
    elif score1 > score2:
        D[team1] += 3
        D[team2] += 0
    elif score1 < score2:
        D[team1] += 0
        D[team2] += 3

return max(D.values()) - min(D.values())

```

Test your solution

```

# Test case 1 (Example)
teams = ["Team A", "Team B", "Team C"]
results = [("Team A", "Team B", 2, 1), ("Team B", "Team C", 0, 3), ("Team A", "Team C", 3, 0)]
result = my_function(teams, results)
assert result == {"Team A": 4, "Team B": 0, "Team C": 4}

# Test case 2
teams = ["Team X", "Team Y"]
results = [("Team X", "Team Y", 3, 0)]
result = my_function(teams, results)
assert result == {"Team X": 3, "Team Y": 0}

```

```

# Test case 3
teams = ["Team P", "Team Q", "Team R"]
results = [("Team P", "Team Q", 1, 1), ("Team Q", "Team R", 2, 0), ("Team P", "Team R", 3, 0)]
result = my_function(teams, results)
assert result == {"Team P": 1, "Team Q": 1, "Team R": 3}

# Test case 4
teams = []
results = []
result = my_function(teams, results)
assert result == {}

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 38: Morse Code

Exercise Description

Write a function `my_function(text)` that converts text to Morse code.

The function should convert each letter in the input text to its corresponding Morse code representation and return the result as a string with Morse code symbols separated by spaces.

Use the standard International Morse Code alphabet. If the input is empty, return an empty string.

Your solution

```
def my_function(s, morse_dict):  
    """  
    Define a function my_function(s, morse_dict) that decodes a string s, representing  
    In Morse Code:  
    * each letter is encoded as a sequence of dashes and dots. For example, letter C  
    * encoded letter are separated by a / symbol (slash)  
    Parameter morse_dict is a dictionary whose keys are morse-encoded letters: keys a  
    The function shall return the decoded string or an empty string in case s is empty  
    """  
    if s == '':  
        return s  
    splitted = s.split('/')  
    decoded = ''  
    for item in splitted:  
        decoded = decoded + morse_dict[item]  
    return decoded
```

Test your solution

```
# Test case 1 (Example)
result = my_function("HELLO")
assert result == "... .-. .-. -.-"

# Test case 2
result = my_function("SOS")
assert result == "... -.- ..."

# Test case 3
result = my_function("A")
assert result == "._"

# Test case 4
result = my_function("")
assert result == ""
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 39: Musical Albums

Exercise Description

Write a function `my_function(albums, year)` that filters musical albums by release year.

The `albums` parameter is a list of tuples, where each tuple represents an album: `(title, artist, year, tracks)`.

The function should return a list containing only the albums that were released in the specified year, maintaining the original order.

Your solution

```
def my_function(L, S):  
    # check if L is empty, if so return 0  
    if L == []:  
        return 0  
    # take all albums from L that have genre S  
    subset = [album for album in L if album["genre"] == S]  
    # check if subset is empty, if so return -1  
    if subset == []:  
        return -1  
    # take the average rating of all albums in subset  
    avg_rating = sum([album["rating"] for album in subset]) / len(subset)  
    # return the average rating  
    return avg_rating
```

Test your solution

```

# Test case 1 (Example)
albums = [("Album A", "Artist 1", 2020, 10), ("Album B", "Artist 2", 2019, 8)]
result = my_function(albums, 2019)
assert result == [("Album B", "Artist 2", 2019, 8)]

# Test case 2
albums = [("Album X", "Artist X", 2021, 12), ("Album Y", "Artist Y", 2021, 9)]
result = my_function(albums, 2021)
assert result == [("Album X", "Artist X", 2021, 12), ("Album Y", "Artist Y", 2021, 9)]

# Test case 3
albums = [("Album P", "Artist P", 2018, 7)]
result = my_function(albums, 2020)
assert result == []

# Test case 4
albums = []
result = my_function(albums, 2022)
assert result == []

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 40: Planet 4z Mission Cost

Exercise Description

Write a function `my_function(supplies, costs)` that calculates the total cost of supplies for a mission to Planet 4z.

Parameters:

- `supplies`: A dictionary mapping supply names to quantities needed
- `costs`: A dictionary mapping supply names to cost per unit

The function should return the total cost by multiplying the quantity of each supply by its unit cost and summing all costs.

If a supply is needed but has no cost information, or if a cost is given for a supply that's not needed, ignore it in the calculation.

Your solution

```
def my_function(humans):  
    cost = 0  
    for key in humans:  
        if len(key) <= 4 or "z" in key:  
            age, weight = humans[key]  
            cost += 15  
            if weight > 80:  
                cost += 10
```



```
        if age > 80:
            cost += 5
    return cost
```

Test your solution

```
# Test case 1 (Example)
supplies = {"water": 100, "food": 50, "fuel": 200}
costs = {"water": 10, "food": 20, "fuel": 5}
result = my_function(supplies, costs)
assert result == 3000

# Test case 2
supplies = {"oxygen": 150, "tools": 25}
costs = {"oxygen": 8, "tools": 40}
result = my_function(supplies, costs)
assert result == 2200

# Test case 3
supplies = {}
costs = {"item": 10}
result = my_function(supplies, costs)
assert result == 0

# Test case 4
supplies = {"item1": 10, "item2": 20}
costs = {}
result = my_function(supplies, costs)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 41: Shopping By Quantity

Exercise Description

Shopping by Quantity

Write a function `my_function(cart, inventory)` that processes a shopping cart based on available inventory quantities.

Parameters:

- `cart`: A dictionary mapping item names to desired quantities
- `inventory`: A dictionary mapping item names to available quantities

The function should return a dictionary showing the actual quantities that can be purchased (limited by inventory availability).

Your solution

```
def my_function(prices, cart):
    if len(cart) == 0:
        return 0

    items = [prices[i][0] for i in range(len(prices))]

    s = 0
    for k, v in cart.items():
        i = items.index(k)
        s = s + v * prices[i][1]
    return s
```

Test your solution

```
# Test case 1 (Example)
a = my_function([('cookies', 4.2), ('pasta', 3.0), ('eggs', 3.2), ('milk', 2.3)], {'p
# assert a == expected_result # TODO: Add expected result

# Test case 2
a = my_function([('cookies', 4.2), ('orange', 1.0), ('eggs', 3.2), ('milk', 2.3)], {'
# assert a == expected_result # TODO: Add expected result

# Test case 3
a = my_function([('cookies', 4.2), ('orange', 1.0), ('bacon', 3.2), ('milk', 2.3)], {'
# assert a == expected_result # TODO: Add expected result
```

```
# Test case 4
a = my_function([('cookies', 4.2), ('orange', 1.0), ('bacon', 3.2), ('milk', 2.3)], {
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 42: Shopping By Prices

Exercise Description

Shopping by Prices

Write a function `my_function(cart, prices)` that calculates the total cost of items in a shopping cart.

Parameters:

- `cart`: A dictionary mapping item names to quantities
- `prices`: A dictionary mapping item names to unit prices

The function should return the total cost by multiplying the quantity of each item by its unit price and summing all costs.

Your solution

```
def my_function(shopping_cart, prices):
    total = 0
    if len(shopping_cart) == 0:
        return total
    for object, quantity in shopping_cart:
        total = total + quantity * prices[object]
    return total
```

Test your solution

```
# Test case 1 (Example)
a = my_function([('pasta', 3), ('milk', 2), ('cookies', 4)], {'pasta': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'orange': 0.69})
# assert a == expected_result # TODO: Add expected result

# Test case 2
a = my_function([('orange', 1), ('cookies', 10)], {'cookies': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'orange': 0.69})
# assert a == expected_result # TODO: Add expected result

# Test case 3
a = my_function([('orange', 1)], {'cookies': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'orange': 0.69})
# assert a == expected_result # TODO: Add expected result

# Test case 4
a = my_function([], {'cookies': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'orange': 0.69})
# assert a == expected_result # TODO: Add expected result
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 43: Wallet

Exercise Description

Write a function `my_function(wallet, price)` that computes the total value inside the wallet in Euro.

Parameter `wallet` is a list of tuples `(currency, quantity)`, where `currency` is a string and `quantity` is a number: for example, `wallet` could be `[("gbp", 3), ("dollar", 2), ("gbp", 4)]`.

Parameter `price` is a dictionary that maps the conversion of different currencies in euro (1 euro), for example `{"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65}`.

You can assume that the currency in each item in the wallet appears as a key in the price dictionary.

The function shall return the total value in euro inside the wallet. If there are no items in the wallet, the function should return 0.

Back-of-the-envelope example: `my_function([("gbp", 2), ("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65})` should return $4.22 = 2 \cdot 0.89 + 2 \cdot 1.22$.

Your solution

```
def my_function(wallet, price):
    if wallet == []:
        return 0
    else:
        s = 0
        for currency, quantity in wallet:
            s += quantity * price[currency]
        return s
```

Test your solution

```
# Test case 1 (Example)
a = my_function([("gbp", 2), ("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65})
# assert a == expected_result # TODO: Add expected result

# Test case 2
a = my_function([("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65})
# assert a == expected_result # TODO: Add expected result
```

```
# Test case 3
a = my_function([("krona", 1), ("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65})
# assert a == expected_result # TODO: Add expected result

# Test case 4
a = my_function([], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65})
# assert a == expected_result # TODO: Add expected result
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 44: Automatic writing

Exercise Description

Write a function `my_function(a, b)` that takes two lists:

- `a` is a list of characters, for example `['b', 'e', 'l', 'a']`
- `b` is a list of integers, for example `[1, 1, 2, 1]`

The function should behave as follows:

- If the two lists a and b have different lengths, the function returns -1.
- If there is at least one negative value in b, the function returns -1.
- Otherwise, the function returns a string containing the characters in a repeated as many times as the number specified in the corresponding position in b.

For example, `my_function(['b','e','l','a'], [1,1,2,1])` should return 'bella'.

Your solution

```
def my_function(a, b):  
    # check whether the two lists have different length: if so, return -1  
    if len(a) != len(b):  
        return -1  
    # check whether there are some negative elements in b: if so, return -1  
    for n in b:  
        if n < 0:  
            return -1  
    # if no returns have been triggered so far, then create the proper string  
    s = ''  
    for i in range(len(a)):  
        s += a[i] * b[i]  
    return s
```

Test your solution

```
# Test case 1 (Example)  
assert my_function(['b','e','l','a'], [1,1,2,1]) == 'bella'
```

```
# Test case 2
assert my_function(['a', 'b', 'c'], [3,2,0]) == 'aaabb'

# Test case 3
assert my_function(['a', 'b', 'c'], [3,2,-1]) == -1

# Test case 4
assert my_function(['a', 'b', 'c'], [3,2,2,2]) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 45: List concatenations even n

Exercise Description

Write a function `my_function(lst, n)` which, given as input a list of strings `lst` and a number `n`, returns a string that concatenates all the strings in the list that:

- have length strictly greater than n , and
- have an even length.

If none of the strings satisfy the requirements, the function must return the string 'no'.

If the input list is empty, the function must return -1.

Your solution

```
def my_function(lst, n):  
    if len(lst) == 0:  
        return -1  
    result = ""  
    for el in lst:  
        if len(el) > n and len(el) % 2 == 0:  
            result += el  
    if len(result) == 0:  
        return "no"  
    else:  
        return result
```

Test your solution

```
# Test case 1 (Example)  
assert my_function(["chirus", "sam", "z", "alessio martino"], 4) == "chirus"  
  
# Test case 2  
assert my_function(["chirus", "samo", "zo", "alessio martino", "time"], 2) == "chirus"  
  
# Test case 3
```

```
assert my_function(["chirus", "samo", "zo", "alessio martino", "time"], 22) == "no"

# Test case 4
assert my_function([], 4) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 46: Loop Game 1: prime-weighted sum

Exercise Description

Write a function `my_function(lst)` which takes as input a list of numbers `lst` and returns the sum of the numbers in the list, with the following rules:

- Prime numbers are worth double (for example, 7 is worth 14).
- If a 0 is encountered, the current sum is reset to 0.
- If the number -1 is encountered, the loop is interrupted and the function returns the sum obtained up to that point.

If the list is empty, the function should return 0.

Your solution

```
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

def my_function(lst):
    result = 0
    for el in lst:
        if el == -1:
            break
        elif el == 0:
            result = 0
        elif is_prime(el):
            result += el * 2
        else:
            result += el
    return result
```

Test your solution

```
# Test case 1 (Example)
assert my_function([4, 6, 8]) == 18

# Test case 2
assert my_function([99, 45, 18, 0, 7]) == 14
```

```
# Test case 3
assert my_function([40, 40, -1, 100, 7, 0, 12]) == 80

# Test case 4
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 47: Loop Game 2: character counter

Exercise Description

Write a function `my_function(lst)` which takes as input a list of strings `lst` and returns the total "value" of all the characters in the list. The rules are:

- Lowercase letters are worth 1 point each.
- Uppercase letters are worth 2 points each.
- If the character '0' is encountered, the current total is doubled (the character '0' itself does not add points).
- If the lowercase character 'z' is encountered, the loop stops immediately and the function returns -1.

The function returns the final value according to these rules.

Your solution

```
def my_function(lst):  
    capital = "ABCDEFGHILMNOPQRSTUVWXYZ"  
    result = 0  
    for el in lst:  
        for c in el:  
            if c in capital:  
                result += 2  
            elif c == "0":  
                result += result  
            elif c == "z":  
                return -1  
            else:  
                result += 1  
    return result
```

Test your solution

```
# Test case 1 (Example)
assert my_function(["hello", "test", "hi"]) == 11

# Test case 2
assert my_function(["hEllo", "teSt", "hi"]) == 13

# Test case 3
assert my_function(["hEllo", "teSt", "0i"]) == 23

# Test case 4
assert my_function(["hEllz", "teSt", "0i"]) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 48: Loop Game 3: even numbers and powers

Exercise Description

Write a function `my_function(lst)` which takes as input a list of numbers `lst` and returns the sum of the numbers in the list, with the following rules:

- Even numbers less than 100 are worth double (for example, 50 is worth 100).
- If a 0 is encountered, the current sum is doubled.
- If the number -1 is encountered, the loop is interrupted and the function returns the square of the current sum (sum ** 2).

If the list is empty, the function should return 0.

Your solution

```
import math

def my_function(lst):
    result = 0
    for el in lst:
        if el % 2 == 0 and el < 100:
            result += el * 2
        elif el == 0:
            result += result
        elif el == -1:
            return math.pow(result, 2)
        else:
            result += el
    return result
```

Test your solution

```
# Test case 1 (Example)
assert int(my_function([1, 1, 1, 102])) == 105
```

```
# Test case 2
assert int(my_function([2, 2, 2, 102])) == 114

# Test case 3
assert int(my_function([])) == 0

# Test case 4
assert int(my_function([2, 2, -1, 102])) == 64
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 49: Loop Game 4: filtered strings count

Exercise Description

Write a function `my_function(lst, n)` which takes as inputs a list of strings `lst` and an integer `n` and returns the number of elements in the list that satisfy all the following conditions:

- the string length is **less than or equal to** `n`, and
- the string does **not** contain any uppercase characters.

If an empty string `"` is encountered in the list, the function must stop and return the count obtained up to that point.

Your solution

```
def checkUpper(w):  
    capital = "ABCDEFGHILMNOPQRSTUVWXYZ"  
    for el in w:  
        if el in capital:  
            return True  
    return False  
  
def my_function(lst, n):  
    count = 0  
    for el in lst:  
        if el == "":  
            return count  
        if (not checkUpper(el)) and len(el) <= n:  
            count += 1  
    return count
```

Test your solution

```
# Test case 1 (Example)  
assert my_function(["a", "ab", "abc"], 3) == 3
```

```
# Test case 2
assert my_function(["a", "ab", "abc"], 2) == 2

# Test case 3
assert my_function(["A", "aB", "abc"], 3) == 1

# Test case 4
assert my_function(["abcd", "ab", "A", "", "abc"], 3) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 50: Merge words

Exercise Description

Write a function `my_function(a, b)` that adds two lists index-wise.

- a and b are lists whose elements are strings (or characters).
- The function returns a new list where each element is the concatenation of the elements from a and b in the same position.

If the lists do not have the same length, the function should return -1.

Your solution

```
def my_function(a, b):  
    if len(a) != len(b):  
        return -1  
    res = [i + j for i, j in zip(a, b)]  
    return res
```

Test your solution

```
# Test case 1 (Example)  
assert my_function(['Pro', 'Ale', 'Mar'], ['f', 'ssio', 'tino']) == ['Prof', 'Alessio', 'Marino']  
  
# Test case 2  
assert my_function(['Pro', 'Ale', 'Mar'], ['f', 'ssio']) == -1  
  
# Test case 3  
assert my_function(['Ta', 'va', 'gat'], ['nto', ' la', 'ta']) == ['Tanto', 'va la', 'gatt']  
  
# Test case 4  
assert my_function(['Pro', 'Ale'], ['nto', 'ssandro']) == ['Pronto', 'Alessandro']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 51: Password Encoder

Exercise Description

Write a function `my_function(s, letter)` that helps to build a password.

- `s` is a string.
- `letter` is a character and must be one of `['i', 'a', 's']`.

The function converts the string into a password with the following rules:

- If there are **no capital letters** in `s`, the function returns `-1`.
- If letter is **not** one of `'i'`, `'a'`, `'s'`, the function returns `-1`.
- If letter does not appear in `s`, the function returns `-1`.
- Otherwise, it returns a new string where every occurrence of letter is replaced by a special character according to this mapping:
 - `'i'` becomes `'!'`
 - `'a'` becomes `'@'`
 - `'s'` becomes `'$'`

All other characters remain unchanged.

Your solution

```
def my_function(s, letter):  
    capital = "ABCDEFGHILMNOPQRSTUVWXYZ"  
    c = False  
    for x in s:  
        if x in capital:  
            c = True  
    if c is False:  
        return -1  
    if letter not in 'ais':  
        return -1  
    if letter not in s:  
        return -1  
    res = ""  
    for x in s:  
        if x == letter:
```

```
        if letter == 'i':
            res += '!'
        elif letter == 'a':
            res += '@'
        elif letter == 's':
            res += '$'
    else:
        res += x
    return res
```

Test your solution

```
# Test case 1 (Example)
assert my_function("Mammabuttalapasta", "a") == "M@mm@butt@l@p@st@"

# Test case 2
assert my_function("SerpentiSuonatoridiSonagli", "s") == -1

# Test case 3
assert my_function("CaniCattivi", "i") == "Can!Catt!v!"

# Test case 4
assert my_function("CaniCattivi", "q") == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**


```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 52: Planet 4z

Exercise Description

It is the year 2222 and the Earth is nearing its end. The inhabitants of planet 4z have decided to help the earthlings by sending escape ships. Each spacecraft can hold only one person.

On planet 4z there is an inviolable rule: the name of all its inhabitants must **contain the letter 'z'** or be **at most 4 characters long**. Any humans who fail to meet these requirements will not be saved.

Write a function `my_function(inhabitants)` that, given as input a list of names of earthlings, returns the number of ships to be sent to planet Earth (i.e., the number of humans that satisfy the rule).

If none of the names meet the requirements, the function must return `-1`.

Your solution

```
def my_function(inhabitants):  
    ships = 0  
    for el in inhabitants:  
        if len(el) <= 4 or 'z' in el:  
            ships += 1
```

```
if ships == 0:
    return -1
else:
    return ships
```

Test your solution

```
# Test case 1 (Example)
assert my_function(["chirus", "zzzzzzz"]) == 1

# Test case 2
assert my_function(["chirus"]) == -1

# Test case 3
assert my_function(["chirus", "sam", "z", "alessio martino"]) == 2

# Test case 4
assert my_function([]) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 53: Repeat

Exercise Description

Write a function `my_function(lst)` that takes a list of integers as input and returns a new list of integers.

The new list is built as follows:

- the element in position 0 is repeated 1 time,
- the element in position 1 is repeated 2 times,
- the element in position 2 is repeated 3 times, and so on.

However, you do **not** like the number 4, therefore:

- you must **ignore** any element whose value is 4 in the output list, and
- you must **never** repeat an element **4 times** (i.e., when the position index + 1 is 4, skip that element).

Your solution

```
def my_function(lst):  
    res = []  
    for i, val in enumerate(lst):  
        i += 1  
        if i == 4 or val == 4:  
            continue
```

```
        for _ in range(i):
            res.append(val)
    return res
```

Test your solution

```
# Test case 1 (Example)
assert my_function([1, 2, 3, 4]) == [1, 2, 2, 3, 3, 3]

# Test case 2
assert my_function([9, 8, 7, 6, 5]) == [9, 8, 8, 7, 7, 7, 5, 5, 5, 5, 5]

# Test case 3
assert my_function([4, 4, 4, 2, 1]) == [1, 1, 1, 1, 1]

# Test case 4
assert my_function([4, 4, 4, 4, 4, 4, 4]) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 54: Reverse (almost) everything

Exercise Description

Write a function `my_function(lst, key)` that takes as input a list of words `lst` and a word `key`.

- If `key` is **not** in the list, the function should return `-1`.
- Otherwise, it should return a new list where **all words are reversed**, except for `key`, which remains unchanged (but stays in its original position).

Your solution

```
def my_function(lst, key):  
    if key not in lst:  
        return -1  
    res = []  
    for x in lst:  
        if x != key:  
            res.append(x[::-1])  
        else:  
            res.append(x)  
    return res
```

Test your solution

```
# Test case 1 (Example)
assert my_function(['hello', 'world', 'python'], 'world') == ['olleh', 'world', 'noht']

# Test case 2: key not present
assert my_function(['a', 'b', 'c'], 'x') == -1

# Test case 3: key at the beginning
assert my_function(['key', 'abc', 'def'], 'key') == ['key', 'cba', 'fed']

# Test case 4: key at the end
assert my_function(['abc', 'def', 'key'], 'key') == ['cba', 'fed', 'key']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML; import urllib.parse as u; HTML('<a href="https://pyth
```

Exercise 55: Sandwich

Exercise Description

Write a function `my_function(lst, k)` that returns a list of all the elements in `lst` that are **between two values** `k`.

That is, for each element in the list, you check whether the element before it and the element after it are both equal to k . If so, this element is included in the output.

Additional rules:

- If there are multiple occurrences of the same value that satisfy the condition, the output list should contain each such value **only once** (no duplicates).
- If k is not in `lst`, the function should return `-1`.

Your solution

```
def my_function(lst, k):
    if k not in lst:
        return -1
    res = []
    for i, v in enumerate(lst[:-1]):
        if v in res or i == 0:
            continue
        if (lst[i - 1] == k) and (lst[i + 1] == k):
            res.append(v)
    return res
```

Test your solution

```
# Test case 1 (Example)
assert my_function([2, 1, 2, 3, 4, 5, 2, 4, 2, 4, 2], 2) == [1, 4]

# Test case 2: k not present
```

```
assert my_function([2, 1, 2, 3, 4, 5, 2, 4, 2, 4, 2], 7) == -1

# Test case 3: repeated values
assert my_function([2, 2, 2, 3, 4, 5, 2, 4, 2, 4, 2], 2) == [2, 4]

# Test case 4
assert my_function([2, 2, 2, 2, 2, 2, 2, 3, 4, 3, 4, 3, 2, 3, 2], 2) == [2, 3]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 56: Converting Time 1

Exercise Description

Write a function `my_function(K, L)` that is given an integer number `K` and a list of tuples `L`. Each tuple is a pair consisting of an integer number and a character `'m'` (minutes), `'h'` (hours), or `'s'` (seconds).

After converting hours and minutes to seconds, the function should return the position `i` of the tuple in list `L` such that the sum of seconds up to position `i` is **smaller than or equal**

to K. Positions start from 0.

If the input list is empty or the number of seconds specified by the tuple in position 0 is larger than K, the function should return -1.

Example: `my_function(1000, [(2, 'm'), (25, 's'), (1, 'h'), (1, 's')])` should return 1, because 2 minutes = 120 seconds, 1 hour = 3600 seconds, and $120 + 25 = 145 < 1000$, while $120 + 25 + 3600 = 3745 > 1000$.

Your solution

```
def my_function(K, L):
    L_sec = []
    for i, unit in L:
        if unit == 's':
            L_sec.append(i)
        elif unit == 'm':
            L_sec.append(i * 60)
        elif unit == 'h':
            L_sec.append(i * 3600)

    if len(L) == 0 or L_sec[0] > K:
        return -1

    cumsum = [0 for _ in range(len(L))]
    for i in range(len(L)):
        cumsum[i] = sum(L_sec[: i + 1])

    TF = [item <= K for item in cumsum]
```

```
if False in TF:
    return TF.index(False) - 1
else:
    return len(L) - 1
```

Test your solution

```
# Test case 1 (Example)
assert my_function(1000, [(2, 'm'), (25, 's'), (1, 'h'), (1, 's')]) == 1

# Test case 2
assert my_function(121, [(2, 'm'), (25, 's'), (1, 'h'), (1, 's')]) == 0

# Test case 3
assert my_function(1, [(1, 'm'), (25, 's'), (1, 'h'), (1, 's')]) == -1

# Test case 4
assert my_function(1, []) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 57: Converting Time 2

Exercise Description

Write a function `my_function(K, L)` that is given an integer number `K` and a list of tuples `L`. Each tuple is a triple consisting of three integer numbers, corresponding to **hours, minutes, and seconds**.

After computing the total number of seconds given by each tuple, the function should return the position `i` of the tuple in list `L` such that the sum of seconds up to position `i` is **smaller than or equal to** `K`. Positions start from 0.

If the input list is empty or the number of seconds specified by the tuple in position 0 is larger than `K`, the function should return `-1`.

Example: `my_function(4000, [(0, 2, 3), (1, 3, 5), (2, 0, 32)])` should return 1.

Your solution

```
def my_function(K, L):
    L_sec = []
    for h, m, s in L:
        L_sec.append(s + m * 60 + h * 3600)

    if len(L) == 0 or L_sec[0] > K:
        return -1

    cumsum = [0 for _ in range(len(L))]
```

```

for i in range(len(L)):
    cumsum[i] = sum(L_sec[: i + 1])

TF = [item <= K for item in cumsum]

if False in TF:
    return TF.index(False) - 1
else:
    return len(L) - 1

```

Test your solution

```

# Test case 1 (Example)
assert my_function(4000, [(0, 2, 3), (1, 3, 5), (2, 0, 32)]) == 1

# Test case 2
assert my_function(344000, [(0, 0, 3), (0, 0, 5), (2, 0, 32)]) == 2

# Test case 3
assert my_function(2, [(0, 0, 3), (1, 0, 5), (2, 0, 32)]) == -1

# Test case 4
assert my_function(2, []) == -1

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 58: Divisible by 4

Exercise Description

You have to compute how many numbers divisible by 4 with remainder 1 or 2 you have to sum up until the value of the sum becomes **larger than or equal** to n .

The series of numbers divisible by 4 with remainder 1 or 2 starts with 1, 2, 5, 6, 9, 10, ... (since dividing 1 by 4 gives remainder 1, 2 gives remainder 2, 5 gives remainder 1, and so on).

Write a function `my_function(n)` that gets an integer value n as input and returns the **number of terms** in the smallest sum whose value is $\geq n$. The sum of 0 terms is equal to 0.

Example: if the input is 25, the output is 6 because $1 + 2 + 5 + 6 + 9 = 23 < 25$ but $1 + 2 + 5 + 6 + 9 + 10 = 33 \geq 25$.

Your solution

```
def my_function(n):  
    x = 1  
    mySum = 0  
    i = 0  
    while mySum < n:
```

```
        if x % 4 == 1 or x % 4 == 2:
            mySum = mySum + x
            i = i + 1
        x = x + 1
    return i
```

Test your solution

```
# Test case 1 (Example)
assert my_function(25) == 6

# Test case 2
assert my_function(8) == 3

# Test case 3
assert my_function(75) == 9

# Test case 4
assert my_function(0) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 59: Fitting sum

Exercise Description

You are given an integer number n as input, representing the maximum amount available. Sum all the natural numbers (1, 2, 3, ...) that fit in n , i.e., all natural numbers whose cumulative sum is smaller than or equal to n .

Define a function `last_fit(n)` that returns the **last number** that fits in this sum.

Example: if $n = 12$, the output is 4 because $1 + 2 + 3 + 4 \leq 12$ but $1 + 2 + 3 + 4 + 5 > 12$.

Your solution

```
def last_fit(n):  
    mySum = 0  
    toBeAdded = 0  
    while mySum <= n:  
        toBeAdded = toBeAdded + 1  
        mySum = mySum + toBeAdded  
    return toBeAdded - 1
```

Test your solution

```
# Test case 1 (Example)  
assert last_fit(12) == 4
```

```
# Test case 2
assert last_fit(8) == 3

# Test case 3
assert last_fit(10) == 4

# Test case 4
assert last_fit(100) == 13
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 60: KM CM to M

Exercise Description

Write a function `my_function(S, L)` that is given an integer number `S` and a list of tuples `L`. Each tuple is a pair consisting of an integer number and a character `'k'` (kilometers), `'c'` (centimeters), or `'m'` (meters).

After converting kilometers and centimeters to meters, the function should return the position `i` of the tuple in list `L` such that the sum of meters up to position `i` is **smaller than or equal to** `S`. Positions start from 0.

If the input list is empty or the number of meters specified by the tuple in position 0 is larger than `S`, the function should return `-1`.

Example: `my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')])` should return 1, because $30000 \text{ cm} = 300 \text{ m}$, $5 \text{ km} = 5000 \text{ m}$, and $2 + 300 = 302 \leq 1000$, while $2 + 300 + 5000 = 5302 > 1000$.

Your solution

```
def my_function(S, L):
    if not L or L[0][0] > S:
        return -1

    total = 0
    i = 0
    while i < len(L):
        value, unit = L[i]
        if unit == 'k':
            value *= 1000
        elif unit == 'c':
            value *= 0.01

        total += value
        if total > S:
            return i - 1
```

```
        i += 1
    return i - 1
```

Test your solution

```
# Test case 1 (Example)
assert my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')]) == 1

# Test case 2
assert my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'c')]) == 2

# Test case 3
assert my_function(1, [(20, 'k'), (30000, 'c'), (5, 'k')]) == -1

# Test case 4
assert my_function(1000, []) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 61: Larger in a sequence

Exercise Description

Write a function `first_larger(a, b, n)` that receives three natural numbers `a`, `b`, and `n`.

Starting from `a` and `b`, the function generates a sequence where each element is equal to the sum of the previous two elements. It then returns the **first element in the sequence that is strictly larger than `n`**.

Example: if `a = 1`, `b = 2`, and `n = 50`, the sequence is 1, 2, 3, 5, 8, 13, 21, 34, 55, ... and the function should return 55.

Your solution

```
def first_larger(a, b, n):
    second_to_last = a
    last = b

    while last < n:
        new = last + second_to_last
        second_to_last = last
        last = new

    return last
```

Test your solution

```
# Test case 1 (Example)
assert first_larger(1, 2, 50) == 55
```

```
# Test case 2
assert first_larger(1, 2, 15) == 21

# Test case 3
a = 2; b = 3; n = 8
assert first_larger(a, b, n) == 8

# Test case 4
a = 1; b = 1; n = 1000
assert first_larger(a, b, n) == 1597
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 62: Sum 2 Series

Exercise Description

You have to compute the sum of a series of numbers 2, 22, 222, 2222, and so on, until the value of the sum becomes larger than or equal to n.

Write a function `my_function(n)` that gets an integer value `n` as input parameter and returns the **number of terms** in the smallest sum whose value is $\geq n$. The sum of 0 terms is equal to 0.

Hint: each number can be obtained from the previous one in the series by multiplying it by 10 and adding 2 (e.g., $2222 = 222 * 10 + 2$).

Example: if the input is 25, the output is 3 because $2 + 22 = 24 < 25$ but $2 + 22 + 222 = 246 \geq 25$.

Your solution

```
def my_function(n):
    twos = 2
    mySum = 0
    i = 1
    while mySum < n:
        mySum = mySum + twos
        twos = 10 * twos + 2
        i = i + 1
    return i - 1
```

Test your solution

```
# Test case 1 (Example)
assert my_function(25) == 3

# Test case 2
assert my_function(1000) == 4
```

```
# Test case 3
assert my_function(2468) == 4

# Test case 4
assert my_function(1) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 63: Sum Fermat

Exercise Description

You have to compute the sum of numbers 11, 101, 1001, 10001, 100001, and so on, until the value of the sum becomes larger than or equal to n .

Write a function `my_function(n)` that gets an integer value n as input parameter and returns the **number of terms** in the smallest sum whose value is $\geq n$. The sum of 0 terms is equal to 0.

Hint: each number in the series is a power of 10 plus 1 (e.g., $10001 = 10^4 + 1$).

Example: if the input is 250, then the output is 3 because $11 + 101 = 112 < 250$ but $11 + 101 + 1001 = 1113 \geq 250$.

Your solution

```
def my_function(n):  
    howMany0 = 0  
    mySum = 0  
    while mySum < n:  
        mySum = mySum + int('1' + howMany0 * '0' + '1')  
        howMany0 = howMany0 + 1  
    return howMany0
```

Test your solution

```
# Test case 1 (Example)  
assert my_function(250) == 3  
  
# Test case 2  
assert my_function(11114) == 4  
  
# Test case 3  
assert my_function(3000) == 4  
  
# Test case 4  
assert my_function(1) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 64: Sum Km and Cm to M

Exercise Description

Write a function `my_function(S, L)` that is given an integer number `S` and a list of tuples `L`. Each tuple is a pair consisting of an integer number and a character `'k'` (kilometers), `'c'` (centimeters), or `'m'` (meters).

After converting kilometers and centimeters to meters, the function should return the **largest sum** `s` of meters such that $s \leq S$. The sum must be computed starting from position 0 and using elements in consecutive positions in the list.

If the input list is empty or the number of meters specified by the tuple in position 0 is larger than `S`, the function should return `-1`.

Example: `my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')])` should return 302, because $30000 \text{ cm} = 300 \text{ m}$, $5 \text{ km} = 5000 \text{ m}$, and $2 + 300 = 302 \leq 1000$, while $2 + 300 + 5000 = 5302 > 1000$.

Your solution

```
def my_function(S, L):  
    # check if the list is empty  
    if not L:  
        return -1  
    # check if the first element is greater than S  
    if L[0][1] == 'k' and L[0][0] * 1000 > S:  
        return -1  
    if L[0][1] == 'c' and L[0][0] / 100 > S:  
        return -1  
    if L[0][1] == 'm' and L[0][0] > S:  
        return -1  
  
    sum_meters = 0  
    max_sum = 0  
  
    for distance, unit in L:  
        if unit == 'k':  
            distance *= 1000  
        elif unit == 'c':  
            distance /= 100  
  
        sum_meters += distance  
  
        if sum_meters > S:  
            break  
  
    max_sum = sum_meters
```

```
return max_sum
```

Test your solution

```
# Test case 1 (Example)
assert int(my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')])) == 302

# Test case 2
assert int(my_function(1000, [(2, 'm'), (3, 'm'), (5, 'm')])) == 10

# Test case 3
assert int(my_function(1, [(2, 'm'), (30000, 'c'), (5, 'k')])) == -1

# Test case 4
assert int(my_function(1000, [])) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 65: Sum of Triangular

Exercise Description

You have to compute the sum of triangular numbers 1, 3, 6, 10, 15, 21, ... until the value of the sum becomes larger than or equal to n .

Write a function `my_function(n)` that gets an integer value n as input parameter and returns the **number of terms** in the smallest sum whose value is $\geq n$. The sum of 0 terms is equal to 0.

Hint: the first triangular number in the sequence (position 1) is 1. For $x > 1$, the triangular number at position x is equal to the triangular number at position $x-1$ plus x .

Example: if the input is 25, then the output is 5 because $1 + 3 + 6 + 10 = 20 < 25$ but $1 + 3 + 6 + 10 + 15 = 35 \geq 25$.

Your solution

```
def my_function(n):  
    i = 1  
    last_x = 0  
    x = 0  
    mySum = 0  
    while mySum < n:  
        x = last_x + i  
        last_x = x  
        i = i + 1  
        mySum = mySum + x  
    return i - 1
```

Test your solution

```
# Test case 1 (Example)
assert my_function(25) == 5

# Test case 2
assert my_function(56) == 6

# Test case 3
assert my_function(60) == 7

# Test case 4
assert my_function(1) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 66: Classroom Attendance Tracker

Exercise Description

Imagine you're helping a school modernize its attendance system. Each classroom has a unique ID and a roster of students. The school wants to track attendance daily, marking students as present or absent.

Write a function `update_attendance(current_attendance, attendance_updates)` that:

- `current_attendance` is a dictionary where keys are classroom IDs and values are dictionaries mapping student names to their attendance status ('Present' or 'Absent').
- `attendance_updates` is a list of tuples (`class_id`, `student_name`, `status`) with the updated status.

The function should apply all updates and return the updated dictionary. If a classroom or student does not exist yet, add them.

Your solution

```
def update_attendance(current_attendance, attendance_updates):
    for t in attendance_updates:
        # unpack the tuple
        classID, studentName, attendanceStatus = t
        # if the class ID exists in the dictionary, update the attendance status for
        if classID in current_attendance:
            current_attendance[classID][studentName] = attendanceStatus
        # otherwise, initialize a new class ID to the dictionary and add the student
        else:
            current_attendance[classID] = {studentName: attendanceStatus}
```

```
# job done
return current_attendance
```

Test your solution

```
# Test case 1 (Example)
current_attendance = {
    'Class 1A': {'Alice': 'Present', 'Bob': 'Absent'},
    'Class 2B': {'Charlie': 'Present', 'Daisy': 'Present'},
}

attendance_updates = [
    ('Class 1A', 'Alice', 'Absent'),
    ('Class 2B', 'Eva', 'Present'),
    ('Class 3C', 'Frank', 'Present'),
]

updated_attendance = update_attendance(current_attendance, attendance_updates)
assert updated_attendance == {
    'Class 1A': {'Alice': 'Absent', 'Bob': 'Absent'},
    'Class 2B': {'Charlie': 'Present', 'Daisy': 'Present', 'Eva': 'Present'},
    'Class 3C': {'Frank': 'Present'},
}

# Test case 2
current_attendance = {
    'Class 3A': {'John': 'Present', 'Sarah': 'Absent'},
    'Class 4B': {'Mike': 'Absent', 'Emily': 'Present'},
}

attendance_updates = [
```

```

    ('Class 3A', 'Sarah', 'Present'),
    ('Class 4B', 'Mike', 'Present'),
    ('Class 5C', 'Anna', 'Present'),
]

updated_attendance = update_attendance(current_attendance, attendance_updates)
assert updated_attendance == {
    'Class 3A': {'John': 'Present', 'Sarah': 'Present'},
    'Class 4B': {'Mike': 'Present', 'Emily': 'Present'},
    'Class 5C': {'Anna': 'Present'},
}

# Additional tests adapted from the import notebook follow the same pattern

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 67: Employee Skillset Tracker

Exercise Description

You are assisting the HR department of TechGiant Inc. to manage employees' skillsets.

Write a function `update_employee_skills(current_skills, skill_updates)` that:

- `current_skills` is a dictionary where keys are employee IDs and values are lists of skills.
- `skill_updates` is a list of tuples (`employee_id`, `list_of_skills`, `action`) where `action` is either `'add'` or `'remove'`.

The function should:

- If `action == 'add'`, **add** all skills in the list to that employee's skills (without removing existing ones).
- If `action == 'remove'`, **remove** each listed skill from the employee's skills (if present).
- If an employee ID is not present in `current_skills`, add it.
- If removing skills leaves an employee with no skills, remove that employee from the dictionary.

The function must return the updated dictionary.

Your solution

```
def update_employee_skills(current_skills, skill_updates):  
    for t in skill_updates:  
        # unpack the tuple  
        employeeID, list_of_skills, action = t  
        # if we already have the employee in the dictionary ...
```



```

    if employeeID in current_skills.keys():
        if action == 'add':
            current_skills[employeeID].extend(listOfSkills)
        elif action == 'remove':
            for skill in listOfSkills:
                if skill in current_skills[employeeID]:
                    current_skills[employeeID].remove(skill)
            # otherwise we just add it
        else:
            current_skills[employeeID] = listOfSkills

# let's get rid of employees with no skills
for employeeID in list(current_skills.keys()):
    if len(current_skills[employeeID]) == 0:
        del current_skills[employeeID]

return current_skills

```

Test your solution

```

# Test case 1 (Example)
current_skills = {
    101: ['Python', 'SQL'],
    102: ['Java', 'C++', 'Python'],
    103: ['HTML', 'CSS'],
}

skill_updates = [
    (101, ['Java'], 'add'),
    (104, ['Python', 'JavaScript'], 'add'),
    (102, ['C++'], 'remove'),

```

```

        (103, ['HTML'], 'remove'),
    ]

    updated_skills = update_employee_skills(current_skills, skill_updates)
    assert updated_skills == {
        101: ['Python', 'SQL', 'Java'],
        102: ['Java', 'Python'],
        103: ['CSS'],
        104: ['Python', 'JavaScript'],
    }

# Additional tests follow the same structure as in the import notebook

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 68: Inventory Management in Gridmart

Exercise Description

Gridmart is organized as a grid where each cell stores the quantity of a product category.

Write a function `update_inventory(current_inventory, updates)` that:

- `current_inventory` is a 2D list (list of lists) where each sublist is a row and each element is the quantity of a product category.
- `updates` is a list of tuples (row, column, delta) describing how many products to add (positive) or subtract (negative).

The function should:

- For each update, check if (row, column) is inside the grid. If any update refers to an invalid position, return None.
- Otherwise, apply all updates in order, modifying `current_inventory`.
- If after an update any quantity becomes negative, return -1.
- If all updates are valid and no quantity goes negative, return the updated 2D list.

Your solution

```
def update_inventory(current_inventory, updates):  
    # calculate the size of the grid  
    nRows = len(current_inventory)  
    nCols = len(current_inventory[0])  
    # run it through the updates  
    for t in updates:  
        # unpack the tuple  
        row, column, quantity = t  
        # check if the row number is valid: if not, return None  
        if row < 0 or row >= nRows:
```

```

        return None
    # check if the column number is valid: if not, return None
    if column < 0 or column >= nCols:
        return None
    # if no returns are triggered, this means that the (row, column) pair is valid
    current_inventory[row][column] += quantity
    # do we have a negative number? if so, return -1
    if current_inventory[row][column] < 0:
        return -1
    # job done
    return current_inventory

```

Test your solution

```

# Test case 1 (Example)
current_inventory = [[20, 15, 10], [30, 25, 5], [12, 8, 3]]
updates = [(0, 1, -5), (1, 0, 10), (2, 2, -2)]
assert update_inventory(current_inventory, updates) == [[20, 10, 10], [40, 25, 5], [12, 8, 3]]

# Test case 2: invalid column index
current_inventory = [[40, 30, 20], [25, 35, 45], [15, 10, 5]]
updates = [(0, 2, 5), (1, 3, -15), (2, 0, 10)]
assert update_inventory(current_inventory, updates) is None

# Test case 3: negative quantity after update
current_inventory = [[50, 60], [20, 30], [70, 80]]
updates = [(0, 0, -100), (1, 1, 20), (2, 0, -15)]
assert update_inventory(current_inventory, updates) == -1

# Test case 4
current_inventory = [[100, 200, 300], [400, 500, 600]]

```

```

updates = [(0, 1, -50), (1, 2, 25), (1, 0, -40)]
assert update_inventory(current_inventory, updates) == [[100, 150, 300], [360, 500, 600]]

# Test case 5
current_inventory = [[5, 10, 15], [20, 25, 30], [35, 40, 45], [50, 55, 60]]
updates = [(3, 2, -10), (2, 1, 20), (0, 0, 5)]
assert update_inventory(current_inventory, updates) == [[10, 10, 15], [20, 25, 30], [35, 40, 45], [50, 55, 60]]

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```

Exercise 69: Library Book Tracker

Exercise Description

A library wants to track whether books are currently 'in' or 'out'.

Write a function `update_library(current_status, transactions)` that:

- `current_status` is a dictionary mapping book titles to their status ('in' or 'out').
- `transactions` is a list of tuples (title, status) where status is 'in' or 'out'.

The function should apply each transaction in order, updating or inserting entries in the dictionary, and then return the updated dictionary. If a title does not exist yet, add it.

Your solution

```
def update_library(current_status, transactions):  
    for transaction in transactions:  
        book, status = transaction  
        current_status[book] = status  
    return current_status
```

Test your solution

```
# Test case 1 (Example)  
current_status = {  
    'The Great Gatsby': 'in',  
    'To Kill a Mockingbird': 'out',  
    '1984': 'in',  
}  
  
transactions = [  
    ('To Kill a Mockingbird', 'in'),  
    ('The Catcher in the Rye', 'out'),  
    ('1984', 'out'),  
]  
  
updated_status = update_library(current_status, transactions)  
assert updated_status == {  
    'The Great Gatsby': 'in',  
    'To Kill a Mockingbird': 'in',
```

```
'1984': 'out',  
'The Catcher in the Rye': 'out',  
}
```

Additional tests mirror the import notebook cases

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 70: The Classroom Gradebook

Exercise Description

Mr. Alan maintains a gradebook where each student's test scores are stored in a row.

Write a function `calculate_student_averages(gradebook)` that:

- Receives a 2D list `gradebook`, where each inner list contains the scores of one student.
- Returns a list of tuples `(student_index, average_score)` where `student_index` is the index of the student in `gradebook` and `average_score` is their average rounded to 2 decimal places.

Caveats:

- Students with no scores are represented by an empty inner list: their average is defined to be 0.

Your solution

```
def calculate_student_averages(gradebook):  
    output = []  
    for i in range(len(gradebook)):  
        sumOfGrades = sum(gradebook[i])  
        numOfGrades = len(gradebook[i])  
        if numOfGrades == 0:  
            output.append((i, 0))  
        else:  
            output.append((i, round(sumOfGrades / numOfGrades, 2)))  
    return output
```

Test your solution

```
# Test case 1 (Example)  
gradebook = [[78, 82, 85], [92, 88, 91], [83, 76, 79], [95, 92]]  
assert calculate_student_averages(gradebook) == [(0, 81.67), (1, 90.33), (2, 79.33),
```


Further tests follow the patterns in the import notebook

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 71: The Seating Arrangement Puzzle

Exercise Description

The community hall seating is organized as a 2D grid of 'Occupied' / 'Available' seats.

Write a function `update_seating_arrangement(current_seating, updates)` that:

- `current_seating` is a 2D list where each inner list is a row of seats.
- `updates` is a list of tuples `(row, col, new_status)` where `new_status` is 'Occupied' or 'Available'.

The function should:

- First check that the grid is regular (all rows have the same length). If not, return -1.
- For each update, if (row, col) is inside the grid, update that seat's status. If (row, col) is outside the grid, ignore that update.
- Return the updated 2D list.

Your solution

```
def update_seating_arrangement(current_seating, updates):  
    # calculate the number of rows  
    n = len(current_seating)  
    # calculate the number of columns for each row  
    m = []  
    for i in range(n):  
        m.append(len(current_seating[i]))  
    # do we have an irregular grid?  
    if len(set(m)) > 1:  
        return -1  
    else:  
        m = m[0]  
    # update the grid  
    for update in updates:  
        # unpack the tuple  
        row, col, new_value = update  
        # do row and col fit in the arrangement?  
        if not (row < 0 or row >= n or col < 0 or col >= m):  
            current_seating[row][col] = new_value  
    # job done  
    return current_seating
```

Test your solution

```
# Test case 1 (Example)
current_seating = [
    ['Available', 'Occupied', 'Available'],
    ['Occupied', 'Occupied', 'Available'],
    ['Available', 'Available', 'Available'],
]

updates = [(0, 0, 'Occupied'), (1, 2, 'Occupied'), (2, 1, 'Occupied')]
assert update_seating_arrangement(current_seating, updates) == [
    ['Occupied', 'Occupied', 'Available'],
    ['Occupied', 'Occupied', 'Occupied'],
    ['Available', 'Occupied', 'Available'],
]

# Additional tests mirror the import notebook behaviour
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 72: Count Pairs

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **number of pairs** (x, y) such that:

1. x is a multiple of 3, strictly larger than 0 and smaller than or equal to n .
2. y is a number strictly larger than a and strictly smaller than b .
3. The difference between x and y is at most k (included), i.e. $x - y \leq k$.

Your solution

```
def my_function(n, a, b, k):  
    numPairs = 0  
    for x in range(3, n + 1, 3):  
        for y in range(a + 1, b):  
            if x - y <= k:  
                numPairs = numPairs + 1  
    return numPairs
```

Test your solution

```
# Test case 1 (Example)  
assert my_function(32, 9, 21, 12) == 95  
  
# Test case 2
```

```
assert my_function(32, 3, 21, 12) == 130
```

```
# Test case 3
```

```
assert my_function(32, 3, 21, 3) == 79
```

```
# Test case 4
```

```
assert my_function(2, 3, 21, 3) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 73: Count Pairs Between

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **number of pairs** (x, y) such that:

1. y is an even number strictly larger than 0 and smaller than or equal to n .
2. x is a number strictly larger than a and strictly smaller than b .
3. The product $x * y$ is at most k (included).

Your solution

```
def my_function(n, a, b, k):  
    numPairs = 0  
    for y in range(2, n + 1, 2):  
        for x in range(a + 1, b):  
            if x * y <= k:  
                numPairs = numPairs + 1  
    return numPairs
```

Test your solution

```
# Test case 1 (Example)  
assert my_function(2, 1, 6, 10) == 4  
  
# Test case 2  
assert my_function(2, 10, 60, 100) == 40  
  
# Test case 3  
assert my_function(6, 2, 6, 100) == 9  
  
# Test case 4  
assert my_function(6, 2, 6, 1) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 74: Maxsum Pairs

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **maximum sum** $x + y$ taken among all pairs (x, y) such that:

- y is a multiple of 3, strictly larger than 0 and strictly smaller than n .
- x is a number strictly larger than a and strictly smaller than b .
- The difference between x and y is at least k (included), i.e. $x - y \geq k$.

If no pair satisfies the conditions, the function should return 0.

Your solution

```
def my_function(n, a, b, k):  
    bestSum = 0  
    for y in range(3, n, 3):  
        for x in range(a + 1, b):
```

```
        if x - y >= k:
            if x + y > bestSum:
                bestSum = x + y
    return bestSum
```

Test your solution

```
# Test case 1 (Example)
assert my_function(20, 1, 21, 12) == 26

# Test case 2
assert my_function(3, 1, 21, 12) == 0

# Test case 3
assert my_function(20, 1, 210, 12) == 227

# Test case 4
assert my_function(20, 1, 6, 2) == 8
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```


Exercise 75: Maxsum pairs Odd

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **maximum sum** $x + y$ taken among all pairs (x, y) such that:

1. x is a positive odd number smaller than or equal to n .
2. y is a number strictly larger than a and smaller than or equal to b .
3. The product $x * y$ is at most k (included).

If no pair satisfies the conditions, the function should return 0.

Your solution

```
def my_function(n, a, b, k):  
    bestSum = 0  
    for x in range(1, n + 1, 2):  
        for y in range(a + 1, b + 1):  
            if x * y <= k:  
                if x + y > bestSum:  
                    bestSum = x + y  
    return bestSum
```

Test your solution

```
# Test case 1 (Example)
assert my_function(20, 1, 6, 2) == 3

# Test case 2
assert my_function(2, 1, 6, 2) == 3

# Test case 3
assert my_function(2, 1, 6, 20) == 7

# Test case 4
assert my_function(2, 1, 6, 1) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 76: Difference between main and anti-diagonal

Exercise Description

Let M be an $(n \times m)$ matrix of numbers organized as a list-of-lists with (n) lists where each inner list has length (m) . You can assume that all inner lists have the same size (m)

and you can assume M to be non-empty.

Write a Python function called `my_function(M)` that, given a matrix M, returns the absolute value of the difference between the sum of its main diagonal and the sum of its anti-diagonal.

Keep in mind the following caveats and definitions:

- normally, main diagonal and anti-diagonal are defined for square matrices only (namely matrices whose number of rows is equal to the number of columns): if (n) is not equal to (m), the function must return -1
- the main diagonal of a matrix is defined as the list of entries lying from the top left to the bottom right corner of the matrix
- the anti-diagonal of a matrix is defined as the list of entries lying from the top right to the bottom left corner of the matrix

Your solution

```
# Teacher solution
def my_function(M):
    n = len(M)
    m = len(M[0])

    # is the matrix a square matrix?
    if n != m:
        return -1

    sum_diag = 0
    for i in range(n):
```

```

        sum_diag += M[i][i]

    sum_antidiag = 0
    for i in range(n):
        sum_antidiag += M[i][n-1-i]

    if sum_antidiag > sum_diag:
        return sum_antidiag - sum_diag
    elif sum_antidiag < sum_diag:
        return sum_diag - sum_antidiag
    else:
        return 0

```

Test your solution

```

# Test case 1 (Example)
M = [[1,2,3], [4,5,6], [7,8,9]]
assert my_function(M) == 0

# Additional tests
M = [[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]]
assert my_function(M) == -1

M = [[29, 10, 12, 33, 83, 28, 32, 81, 91, 84],
      [98, 60, 84, 68, 60, 18, 65, 10, 51, 50],
      [20, 15, 33, 33, 64, 37, 82, 50, 90, 45],
      [80, 38, 9, 43, 69, 82, 78, 39, 46, 44],
      [38, 69, 68, 20, 62, 53, 18, 59, 12, 12],
      [52, 14, 48, 3, 72, 64, 74, 45, 28, 97],
      [6, 27, 4, 2, 82, 67, 79, 96, 50, 55],
      [23, 83, 46, 45, 17, 26, 39, 48, 9, 75],

```

```
[7, 58, 84, 63, 98, 25, 50, 78, 35, 30],  
[55, 36, 22, 32, 45, 14, 14, 5, 75, 34]]  
assert my_function(M) == 62
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 77: Replace odd columns with previous max

Exercise Description

Let M be an $n \times m$ matrix of numbers organized as a list-of-lists with n lists where each inner list has length m . You can assume that all inner lists have the same size m and you can assume M to have at least 2 columns.

Write a Python function called `my_function(M)` that, given a matrix M , returns a modified version of M where the values in the odd columns are replaced with the maximum element of the previous even column.

Furthermore:

- if n is not equal to m, the function must return -1
- if n is equal to m, but there is at least one negative element (regardless of its position), the function must return -2

Your solution

```
# Teacher solution
def my_function(M):
    n = len(M)
    m = len(M[0])

    # is the matrix a square matrix?
    if n != m:
        return -1

    # any negative elements?
    for i in range(n):
        for j in range(m):
            if M[i][j] < 0:
                return -2

    maxval = 0
    for j in range(m):
        if j % 2 == 0:
            l = []
            for i in range(n):
                l.append(M[i][j])
            maxval = max(l)
        else:
            for i in range(n):
```

```
return M
```

```
M[i][j] = maxval
```

Test your solution

```
# Test case 1 (Example)
M = [[1,2,3], [4,5,6], [7,8,9]]
assert my_function(M) == [[1, 7, 3], [4, 7, 6], [7, 7, 9]]
```

Test your solution

```
# Additional tests
M = [[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]]
assert my_function(M) == -1

M = [[1,2,3], [-0.01,5,6], [7,8,9]]
assert my_function(M) == -2

M = [[1, 2, 3, 0, 4], [3, 3, 1, 0, 3], [7, 3, 8, 4, 0], [1, 6, 7, 9, 0], [2, 4, 6, 9, 7]]
assert my_function(M) == [[1, 7, 3, 8, 4], [3, 7, 1, 8, 3], [7, 7, 8, 8, 0], [1, 7, 7, 8, 0], [2, 4, 6, 9, 7]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 78: Replace column i with row i

Exercise Description

Write a function `my_function(M, i)` that is given a list of lists of numbers `M` and an integer number `i` such that:

- `M` represents a square matrix containing $D \times D$ elements: you can assume that all its sublists (corresponding to rows) have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row and column index: it is larger than or equal to 0, and strictly smaller than `D`.

The function should modify `M` by replacing the `i`-th column with the `i`-th row and should then return the modified matrix.

Back-of-the-envelope example: if `M = [[1,2,3],[4,5,6],[7,8,9]]` and `i = 1`, the returned matrix should be `[[1,4,3],[4,5,6],[7,6,9]]`.

Your solution

```
# Teacher solution
def my_function(M, i):
    D = len(M)
    M_new = M.copy()
```



```
# get row i
tmp = M[i]
# replace column i
for k in range(D):
    M_new[k][i] = tmp[k]
# done
return M_new
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3],[4,5,6],[7,8,9]],1)
assert a == [[1, 4, 3], [4, 5, 6], [7, 6, 9]]
```

Test your solution

```
# Additional tests
a = my_function([[1,2,3],[4,5,6],[7,8,9]],2)
assert a == [[1, 2, 7], [4, 5, 8], [7, 8, 9]]

a = my_function([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]],0)
assert a == [[1, 2, 3, 4], [2, 6, 7, 8], [3, 10, 11, 12], [4, 14, 15, 16]]

a = my_function([[1, 2], [3, 4]],1)
assert a == [[1, 3], [3, 4]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 79: Replace row i with column j

Exercise Description

Write a function `my_function(M, i, j)` that is given a list of lists `M` of numbers and two integer numbers `i` and `j` such that:

- `M` represents a square matrix containing $D \times D$ elements: you can assume that all its sublists (corresponding to rows) have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row index: it is larger than or equal to 0, and strictly smaller than `D`.
- Integer `j` is a column index: it is larger than or equal to 0, and strictly smaller than `D`.

The function should modify `M` by replacing the i -th row with the j -th column and should then return the modified matrix.

Back-of-the-envelope example: if `M = [[1,2,3],[4,5,6],[7,8,9]]` and `i = 1`, `j = 2`, the returned matrix should be `[[1,2,3],[3,6,9],[7,8,9]]`.

Your solution

```
# Teacher solution
def my_function(M, i, j):
    D = len(M)
    M_new = M.copy()
    # get column j
    tmp = []
    for k in range(D):
        tmp.append(M[k][j])
    # replace
    M_new[i] = tmp
    # done
    return M_new
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3],[4,5,6],[7,8,9]],1,2)
assert a == [[1, 2, 3], [3, 6, 9], [7, 8, 9]]
```

Test your solution

```
# Additional tests
a = my_function([[1,2,3],[4,5,6],[7,8,9]],2,2)
assert a == [[1, 2, 3], [4, 5, 6], [3, 6, 9]]

a = my_function([[1,2],[4,5]],0,1)
assert a == [[2, 5], [4, 5]]
```

```
a = my_function([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]],3,0)
assert a == [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [1, 5, 9, 13]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 80: Replace row i with column i

Exercise Description

Write a function `my_function(M, i)` that is given a list `M` of lists of numbers and an integer number `i` such that:

- `M` represents a square matrix containing $D \times D$ elements: you can assume that all its sublists (corresponding to rows) have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row and column index: it is larger than or equal to 0, and strictly smaller than `D`.

The function should modify M by replacing the i-th row with the i-th column and should then return the modified matrix.

Back-of-the-envelope example: if $M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ and $i = 2$, the returned modified matrix should be $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 3 & 6 & 9 \end{bmatrix}$.

Your solution

```
# Teacher solution
def my_function(M, i):
    M_new = M.copy()
    col_i = [row[i] for row in M]
    M_new[i] = col_i
    return M_new
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3],[4,5,6],[7,8,9]],2)
assert a == [[1, 2, 3], [4, 5, 6], [3, 6, 9]]
```

Test your solution

```
# Additional tests
a = my_function([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]],3)
assert a == [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [4, 8, 12, 16]]

a = my_function([[1, 2], [3, 4]],0)
assert a == [[1, 3], [3, 4]]
```

```
a = my_function([[1, 2], [3, 4]],1)
assert a == [[1, 2], [2, 4]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 81: Sum of i-th row and i-th column

Exercise Description

Write a function `my_function(M, i)` that is given a list of lists of numbers `M` and an integer number `i` such that:

- `M` represents a square matrix containing `D x D` elements: you can assume that all of its sublists have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row and column index: it is larger than or equal to `0`, and strictly smaller than `D`.

The function should return the sum of the elements in the i -th row and the elements in the i -th column. Element in row i and column i should be counted twice.

Back-of-the-envelope example: if $M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 8 \\ 7 & 8 & 0 \end{bmatrix}$ and $i = 2$, the code should return 26 because $7 + 8 + 0 + 3 + 8 + 0 = 26$.

Your solution

```
# Teacher solution
def my_function(M, i):
    row_i = M[i]
    col_i = [row[i] for row in M]
    return sum(row_i) + sum(col_i)
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1, 2, 3],[4, 5, 8],[7, 8, 0]], 2)
assert a == 26
```

Test your solution

```
# Additional tests
a = my_function([[1, 2, 3], [4, 5, 8], [1, 8, 0]], 0)
assert a == 12

a = my_function([[1, 2],[4, 5]], 1)
assert a == 16
```

```
a = my_function([[1, 2, 3, 0], [4, 5, 8, 0], [0, 8, 0, 0], [0, 8, 0, 0]], 0)
assert a == 11
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 82: Sum of items in column range

Exercise Description

Write a function `my_function(L, i, j)` that is given a list `L` of lists of numbers and two integer numbers `i` and `j`. `L` represents a matrix with `R` rows and `C` columns (`R` and `C` are not given as input). You can assume that all the `R` sublists have the same length `C`, with $0 \leq i < C$ and $0 \leq j < C$.

The function shall return the sum of items in columns between column `i` and column `j`, or 0 if the matrix is empty or $i > j$.

Back-of-the-envelope example: `my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],1,2)` should return 68 (2+20+6+3+30+7).

Your solution

```
# Teacher solution
def my_function(L, i, j):
    if i > j or L == []:
        return 0

    R = len(L)

    s = 0
    for r in range(R):
        for c in range(i, j + 1):
            s = s + L[r][c]
    return s
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3,4],[10,20,30,40]],[5,6,7,8]],1,2)
assert a == 68
```

Test your solution

```
# Additional tests
a = my_function([[1,2,3,4],[10,20,30,40]],[5,6,7,8]],0,1)
assert a == 44

a = my_function([[1,2,3,4],[1,2,3,4]],[5,6,7,8]],0,1)
assert a == 17
```

```
a = my_function([[1,2,3,4],[1,2,3,4],[5,6,7,8],[10,20,30,50]],0,2)
assert a == 90
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 83: Sum of items in row range

Exercise Description

Write a function `my_function(L, i, j)` that is given a list of lists of numbers `L` and two integer numbers `i` and `j`. `L` represents a matrix with `R` rows and `C` columns (`R` and `C` are not given as input). You can assume that all the `R` sublists have the same length `C`, with $0 \leq i < R$ and $0 \leq j < R$.

The function shall return the sum of items in rows between row `i` and row `j`, or `0` if the matrix is empty or $i > j$.

Back-of-the-envelope example: `my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],1,2)` should return 126 (10+20+30+40+5+6+7+8).

Your solution

```
# Teacher solution
def my_function(L, i, j):
    if i > j or L == []:
        return 0

    R = len(L)
    C = len(L[0])

    s = 0
    for c in range(C):
        for r in range(i, j + 1):
            s = s + L[r][c]
    return s
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],1,2)
assert a == 126
```

Test your solution

```
# Additional tests
a = my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],0,1)
assert a == 110

a = my_function([[1,2,3,4],[1,2,3,4]],0,1)
assert a == 20
```

```
a = my_function([[1,2,3,4],[1,2,3,4],[5,0,0,10]],0,2)
assert a == 35
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 84: Sum of main diagonal

Exercise Description

Write a function `my_function(L)` that accepts a list of lists `L` of numbers. `L` represents a square matrix and you can assume that all sublists have the same length and that the number of rows equals the number of columns.

The function shall return the sum of items in the top-left to bottom-right diagonal, or 0 if the matrix is empty.

Back-of-the-envelope example: `my_function([[1,2,3],[1,2,3],[1,2,3]])` returns 6 (1+2+3).

Your solution

```
# Teacher solution
def my_function(L):
    n = len(L)
    if n == 0:
        return 0
    s = 0
    for i in range(n):
        s += L[i][i]
    return s
```

Test your solution

```
# Test case 1 (Example)
assert my_function([[1,2,3],[1,2,3],[1,2,3]]) == 6
```

Test your solution

```
# Additional tests
assert my_function([]) == 0
assert my_function([[1,1,1],[1,1,1],[0,0,-2]]) == 0

large_matrix = [
    [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    [21, 22, 23, 24, 25, 26, 27, 28, 29, 30],
    [31, 32, 33, 34, 35, 36, 37, 38, 39, 40],
    [41, 42, 43, 44, 45, 46, 47, 48, 49, 50],
    [51, 52, 53, 54, 55, 56, 57, 58, 59, 60],
]
```

```
[61, 62, 63, 64, 65, 66, 67, 68, 69, 70],  
[71, 72, 73, 74, 75, 76, 77, 78, 79, 80],  
[81, 82, 83, 84, 85, 86, 87, 88, 89, 90],  
[91, 92, 93, 94, 95, 96, 97, 98, 99, 100]  
]  
assert my_function(large_matrix) == 505
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 85: Sum of top-right to bottom-left diagonal

Exercise Description

Write a function `my_function(L)` that accepts a list of lists `L` of numbers. `L` represents a square matrix and you can assume that all sublists have the same length and that the number of rows equals the number of columns.

The function shall return the sum of items in the top-right to bottom-left diagonal, or `0` if the matrix is empty.

Back-of-the-envelope example: `my_function([[1,2,3],[1,2,3],[1,2,3]])` returns 6 because $3 + 2 + 1 = 6$.

Your solution

```
# Teacher solution
def my_function(L):
    D = len(L)
    s = 0
    for i in range(D):
        s = s + L[i][-(i + 1)]
    return s
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3],[1,2,3],[1,2,3]])
assert a == 6
```

Test your solution

```
# Additional tests
a = my_function([[10,10,3],[1,10,3],[10,2,3]])
assert a == 23

a = my_function([[10, 10, 3, 1],[1, 10, 3, 9],[10, 2, 3, 10], [0, 2, 3, 10]])
assert a == 6
```

```
a = my_function([])
assert a == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 86: Matrix transpose

Exercise Description

Let M be an $(n \times m)$ matrix of numbers organized as a list-of-lists with (n) lists where each inner list has length (m) .

Write a function called `my_function(M)` that takes as input a matrix M and returns its transpose.

Keep in mind the following caveats and definitions:

- if the input matrix is empty, the function must return -1
- the input list-of-lists must indeed be a valid matrix where all rows have the same length (m): if this does not hold, the function must return -2
- the transpose of an ($n \times m$) matrix is a new ($m \times n$) matrix where the item in position ((i,j)) of the original matrix lies in position ((j,i)) in the transposed matrix

Your solution

```
# Teacher solution
def my_function(M):
    # is the matrix empty?
    if M == []:
        return -1

    n = len(M)
    m = [len(M[i]) for i in range(n)]

    # do all rows have the same length?
    if len(set(m)) != 1:
        return -2
    else:
        m = m[0]
```

